

The present work was submitted to:

Chair of Process and Data Science
RWTH Aachen University

State-Based Process Monitoring in Traditional Event Logs for Concept Drift Detection

Bachelor Thesis

presented by

Bora Deren
438377

First Examiner: Prof. Dr. Wil M. P. van der Aalst
Second Examiner: Prof. Dr. Sander Leemans

Advisor: Dr. Alessandro Berti

Aachen, October 11, 2026

Acknowledgments

If you want to incorporate it, keep in mind that, according to the official regulations, *independent work* is a very important aspect of writing a thesis. Thus, you can be grateful for the emotional support by your favorite pet, yet we recommend you to be careful if you want to write anything more specific. In general, we suggest to omit this section.

Abstract

The abstract should be a concise summary of what you have done. It should cover, in roughly 1 to 2 sentences per topic:

- The domain, i.e., explain the application domain in which your thesis is applicable.
- The problem, i.e., motivate what the problem is you are solving.
- The method, i.e., describe the method you have proposed, and, compare it, briefly & high level to other approaches. In particular, focus on the benefits of your approach versus the other approach(es)
- The evaluation, i.e., how did you evaluate your method, what results did you obtain and what do the results indicate?

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Statement	5
1.3	Research Questions	6
1.4	Research Goals	7
1.5	Contributions	8
1.6	Thesis Structure	8
2	Related Work	11
2.1	Drift Detection in Control-Flow	11
2.2	Drift Detection Beyond Control-Flow	12
2.3	Clustering-Based Drift Detection	12
2.4	States in Process Mining	12
2.5	Event Logs with Known Drifts	13
2.6	Positioning of This Thesis	14
3	Preliminaries	15
3.1	Foundations	15
3.1.1	Mathematical Preliminaries	15
3.1.2	Event Logs, Cases and Traces	16
3.1.3	Process Trees	19
3.1.4	Calendar Windows	20
3.2	Concept Drift	20
3.3	Scaling and Principal Component Analysis	22
3.4	Clustering	25
3.4.1	k-Means	25
3.4.2	Self-Organizing Maps	26
3.4.3	DBSCAN	27
3.4.4	Clustering Evaluation	28
4	Rheon	33
4.1	Generated Log Schema	33
4.2	Generation of an Event Log	33
4.3	Drifts	36
4.4	Examples	38
4.5	Limitations and Future Work	39

5	State-Based Drift Detection	45
5.1	Feature Extraction	45
5.1.1	Intra-Case Perspective	45
5.1.2	Resource Perspective	45
5.1.3	Inter-Case Perspective	45
5.2	Dimensionality Reduction	45
5.2.1	Intra-Case Perspective	45
5.2.2	Resource Perspective	45
5.2.3	Inter-Case Perspective	45
5.3	Clustering into States	45
5.3.1	Intra-Case Perspective	46
5.3.2	Resource Perspective	46
5.3.3	Inter-Case Perspective	46
5.4	Drift Signals	46
5.4.1	Intra-Case Perspective	46
5.4.2	Resource Perspective	46
5.4.3	Inter-Case Perspective	46
6	Implementation	47
6.1	Kairo	47
6.2	Dashboard	47
7	Evaluation	49
7.1	Experimental Setup	49
7.2	Results	49
7.3	Threats to Validity	50
8	Discussion	51
9	Conclusion	53
A	Some additional stuff	55

List of Figures

1.1	A waiting-time drift that control-flow does not show	4
1.2	Drifts on two different time scales	6
2.1	Transition system with states for remaining time prediction	13
3.1	Example process tree	19
3.2	Example event log with calendar windows	20
3.3	The four drift modes	22
3.4	PCA for two correlated features	24
3.5	Three clustering algorithms in comparison	27
4.1	Workflow of the generation of an event log	36
4.2	Python call that generates the event log L_2	38
4.3	Process trees of the example log with a control-flow drift	39
4.4	Example log with a control-flow drift	40
4.5	Example log with a reassignment drift	41
4.6	Example log with an arrival rate drift	42

List of Tables

3.1	Example event log table	18
3.2	Evaluation of the example clusterings	31
4.1	Schema of a Rheon log	34
4.2	Excerpt of a Rheon log	34
4.3	Parameters of Rheon	35
4.4	The nine drift types of Rheon	37
4.5	Example logs per perspective	38

1 Introduction

First of all, we encourage you to read Prof. van der Aalst’s guide on how to write papers [26].

In this section, you introduce the work to your audience. Even though this is the first chapter of your thesis, you should not necessarily write this chapter first. It is often easier to write this chapter at the end of the thesis writing. In this part of your introduction, i.e., the part before the Motivation section (Section 1.1), you write a paragraph (not just a sentence) for each of the bullet points mentioned in the abstract:

- Domain, i.e., explain the application domain in which your thesis is applicable.
- Problem, i.e., motivate what the problem is you are solving.
- Method, i.e., describe the method you have proposed, and, compare it, briefly & high level to other approaches. In particular, focus on the benefits of your approach versus the other approach(es)
- Evaluation, i.e., how did you evaluate your method and what do the results indicate?

Optionally, you can finalize this part with a little overview of what is still to come in this chapter: “The remainder of this chapter is structured as follows. In Section 1.1, we motivate the need for ... In Section 1.2, we provide a formal problem statement. ...”

Before we dive into the motivation section, a few general guidelines for writing:

1. Write *active* and in the *present tense*.
2. Avoid “optionality” as much as possible, i.e., usage of the following words should (pun intended) be minimized:
 - could
 - would
 - should
 - may
 - might
 - can
 - will
 - ...

1 Introduction

Good: We present a method that ...

Bad: We will present a method that ...

Good: We use function f and compute its inverse.

Bad: We can use the function f and will compute its inverse.

3. Do not use `\\` to generate line breaks, but an empty line
4. Position the caption of a Figure below the figure.
5. Position the caption of a Table above the table.
6. Write Section Headings and Titles Like This
Good: Approximation Bias in Unstable Systems
Bad: Approximation bias in unstable systems
7. Be consistent in your references.
 - Make sure that the name of the same author is always the same (using DBLP helps for this)
 - Titles should be consistent. Either use the style previously described, i.e., Approximation Bias in Unstable Systems or Approximation bias in unstable systems (this is allowed in the references, opposed to your own section headings and titles, however, *BE CONSISTENT*).

1.1 Motivation

Played-out event logs do not change, but real processes do. Following van der Aalst [25], the situation in which a process is changing while being analyzed is called a concept drift. For example, two activities are concurrent at the start of an event log but then become sequential. According to van der Aalst [25], typical triggers of concept drift are seasonal effects, such as higher demand in December, and changed market conditions. Classical process mining techniques mostly assume that the process at the start and at the end of the log is the same [8]. If a drift occurs, process models or Key Performance Indicators (KPIs) computed over the entire log mix two different processes. The process owner must therefore detect whether and when the process changed, and what changed.

A drift does not restrict itself to the order of activities. Processes run in a context, but process analysis sometimes ignores it. As described by van der Aalst [25], process context covers how cases influence each other like competing for the same resources. A single case is therefore not sufficient to explain the observed behavior alone. To explain the observed behavior and its changes, we analyze dependencies within cases, between cases, and with resources. For these dependencies, Senderovich et al. [24] introduce the terms intra-case and inter-case dependencies. This thesis adopts this terminology and considers a process from three perspectives:

- *Intra-case perspective*: The history of a single case up to a specific time, covering which activities happened in which order and how often. Drift example: A new trace variant dominates.

- *Resource perspective:* The working style of resources, such as their workload, service times, handovers, and the activities they perform. Drift example: A resource no longer exists and other resources take over the work.
- *Inter-case perspective:* The current situation of all running cases of the process, covering arrivals, waiting times, and case attributes. Drift example: The mean waiting time between consecutive events of a case increases.

From all three perspectives concept drifts can take place, but they can also influence each other. For example, a new trace variant with a frequent activity becomes dominant (intra-case perspective). The resource responsible for that activity then has more work to do (resource perspective), which affects the waiting times in the entire process (inter-case perspective).

We describe each drift by three properties:

- *Drift type:* What changes, e.g., the control-flow, the reassignment of an activity to another resource, or the waiting time between consecutive events.
- *Drift perspective:* The perspective to which the drift type belongs. For example, control-flow drifts belong to the intra-case perspective, reassignment drifts to the resource perspective, and waiting time drifts to the inter-case perspective.
- *Drift mode:* How the change takes effect over time, e.g., as a sudden, gradual, or recurring change.

In the literature, the terms drift type, drift mode, and drift perspective are used interchangeably. We reserve the term drift type for what changes, the term drift mode for how it changes, and the term drift perspective for the group a drift type belongs to.

Existing concept drift detection approaches cover this picture only partially:

- *Control-flow focus:* Most of the work on drift detection focuses on control-flow. Out of the 38 drift detection papers in the survey of Sato et al. [23], only six focus on the time perspective, three on the data perspective, and none on the resource perspective. In our framework, waiting times and case attributes fall under the inter-case perspective, and service times under the resource perspective. A concept drift that only changes the time or resource perspective stays undetected with control-flow approaches.
- *Single KPIs:* Approaches beyond control-flow often track single KPIs. Adams et al. [3] turn each KPI chosen by the user into a time series over time windows and detect drifts in each series separately. A drift therefore shows up as separate change points in several KPIs. This approach does not deliver a compact description of the overall process situation. For steady-state detection, Kraus et al. [16] show that combining complementary process characteristics is more accurate than relying on a single one.

1 Introduction

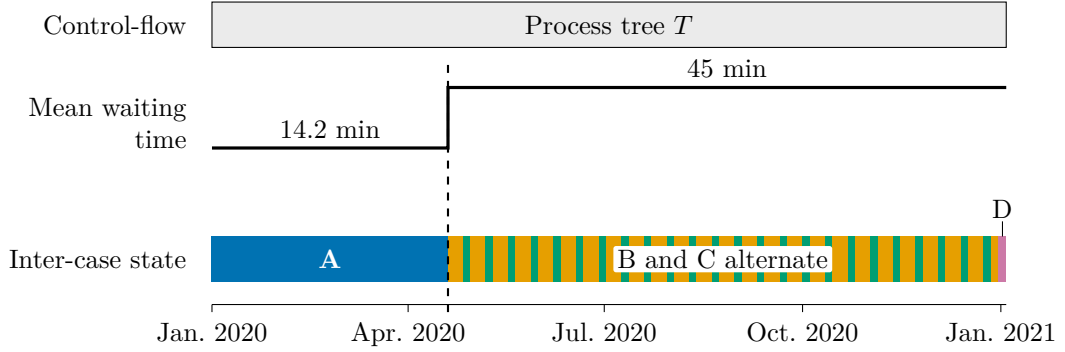


Figure 1.1: Visualization of the synthetic Rheon log generated with a mean waiting time drift. At the drift point (dashed), the mean waiting time between consecutive events of a case rises from 14.2 to 45 min, while all cases still follow process tree T . The inter-case state of each 12-hour window switches exactly there from A to an alternation of B and C. State D marks the run-out of the log after the last case arrivals. The alternation of B and C is drawn schematically.

- *Drift modes:* How a change takes effect over time is also important information for the process owner. Most of the existing approaches focus on sudden drifts only [23].
- *Missing test data:* Sato et al. [23] argue that real-world logs are unsuitable for measuring detection accuracy, since their drift points are unknown. They find only three public synthetic datasets with injected drifts. These datasets cover only the control-flow and time perspectives and do not mix different drift modes. None of them contains a resource drift or a change in case arrivals. A recent benchmark by Padella and Vinci [21] injects drifts into five real logs, but again only into the control-flow and activity durations.

Figure 1.1 shows this gap using a synthetic event log generated by Rheon. Rheon is our generator for synthetic event logs with injected concept drifts in all three perspectives (Chapter 4). This log has 300,299 cases with 1,946,540 events, seven activities, and seven resources. Its cases arrive between January 1 and December 31, 2020. On April 19, 2020, the mean waiting time between consecutive events of a case jumps from 14.2 minutes to 45 minutes. The activities, the control-flow, the resources, and the dominant resource of each activity remain unchanged. A detector that only compares relations between activities sees the same control-flow before and after the drift point and misses the change.

The state-based approach of this thesis aims to detect drifts in all three perspectives and to show what changed. Instead of separate change points per KPI, our states combine many features into one compact description. Rheon provides test logs with known drift points in all three perspectives. In Berti et al. [6], execution states from object-centric event logs are computed for objects. We adapt the approach from objects

to cases in our case-centric framework and define our intra-case states. As an extension to their work, our approach also takes into account resource and inter-case perspectives and adapts the idea to them.

Overall, we compute states from an event log by extracting features, compressing, and clustering. The input of our approach is an event log, and the output consists of intra-case, resource, and inter-case states whose changes over time indicate drifts. In Figure 1.1 one can visually see the general idea of our approach for the inter-case perspective. We describe each 12-hour window by features such as the number of active cases, arrivals, completions, and the mean waiting time between consecutive events of a case. Following the execution states of Berti et al., a Principal Component Analysis (PCA) compresses these features. A 2×2 Self-Organizing Map (SOM) is trained on these compressed feature vectors. Each of its four cells is one state, and each window gets the state of its best-matching cell.

The pipeline is unsupervised and does not see the drift point or the drift type. As seen in Figure 1.1, all 219 windows before the drift point fall into state A. From the drift point until December 31, the windows alternate between states B and C. The last seven windows, in which the log runs out, form state D. The first principal component explains 51% of the variance and is dominated by the mean waiting time between consecutive events of a case. It therefore also shows what changed: after the drift point, more time passes between the events of a case.

1.2 Problem Statement

An event log records a process over a long period, and the process often changes within this period. Such a concept drift affects the intra-case, resource, or inter-case perspective, or several of them at once. A detector that only captures control-flow drifts therefore misses drifts that only affect the resource or inter-case perspective.

Van der Aalst [25] names three main challenges in dealing with concept drifts:

1. *Change point detection*: Did the process change? If yes, when did it change?
2. *Change localization and characterization*: What has changed?
3. *Change process discovery*: How to capture and predict second-order dynamics?

According to van der Aalst, a process is not static even when there is no drift (first-order dynamics). This makes it even harder to detect changes of the process itself (second-order dynamics). Most approaches compare sliding windows of events with statistical methods. To localize drift points exactly and detect them at different time scales, the window length must vary. Moreover, drifts at different time scales can overlap, as in Figure 1.2.

This thesis focuses on challenges 1 and 2. To characterize a drift, our framework determines its drift perspective, its drift mode, and what exactly changed.

In conclusion, this thesis focuses on the following problem:

1 Introduction

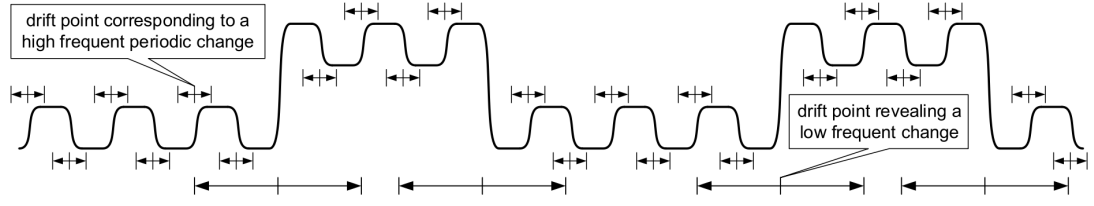


Figure 1.2: A periodically changing process with drifts at two different time scales, taken from van der Aalst [25]. Shorter sliding windows are used to detect short-term drifts. The left half of each sliding window is compared with the right half to spot statistically relevant changes. The larger sliding windows at the bottom are used for long-term drifts.

- *Given:* An event log in which each event belongs to exactly one case. Each event carries a case identifier, an activity, and a timestamp, and optionally a resource and case attributes. Drift points, drift types, drift perspectives, and drift modes are unknown.
- *Objective:* Compute intra-case, resource, and inter-case states whose changes over time indicate drifts. From these changes, detect the drift points and determine for each drift its perspective, its mode, and what changed.
- *Evaluation:* We compare the detected drift points with known drift points in two sets of logs. First, synthetic Rheon logs contain drifts in all three perspectives. Second, the benchmark of Padella and Vinci [21] injects sudden, gradual, and recurring control-flow and duration drifts into five real logs.

1.3 Research Questions

The main research question of this thesis is:

Can a process be monitored as a combination of intra-case, resource and inter-case states, and is this combination a useful indicator for a concept drift in traditional case-centric event logs?

We divide our main research question into five further Research Questions (RQs):

- RQ1** *States:* How can we compute execution states from a traditional event log in the intra-case, resource and inter-case perspectives, and how interpretable are these states in explaining concept drifts?
- RQ2** *Detection:* Do the changes over time in those states signal drift points even if an event log has drifts in multiple perspectives?

- RQ3** *Characterization*: Do the states indicate the perspective and the mode of a drift, and which features suit which drift types?
- RQ4** *Clustering*: Which of the clustering algorithms SOM, k-means and Density-Based Spatial Clustering of Applications with Noise (DBSCAN) delivers the most robust and reliable states?
- RQ5** *Applicability*: In which processes and event logs is the state-based drift detection approach more reliable and when does it deliver deceptive results?

RQ1 addresses the first part of the main research question, monitoring a process through states. RQ2 to RQ5 cover the second part of the main research question, the usage of the states as an indicator for concept drift detection.

1.4 Research Goals

In order to answer the research questions, we follow seven Research Goals (RGs). Each goal lists the research questions it addresses in parentheses.

- RG1** *Terminology*: To clearly define drift type, drift mode and drift perspective (RQ3).
- RG2** *Test data*: Develop a generator for synthetic event logs that injects concept drifts of various drift types in two drift modes (sudden and gradual). Along with each log, the generator writes a metadata file that lists the injected drifts (RQ2, RQ3, RQ5).
- RG3** *Framework*: Develop a state-based framework for concept drift detection (RQ1, RQ2). It adapts execution states from Berti et al. [6] for objects to cases and extends them to resource and inter-case states.
- RG4** *Library*: Develop a Python library with artificial intelligence features that implements our framework (RQ1, RQ4). The library offers each result as a plot or text for Large Language Models (LLMs) and also provides LLM connections.
- RG5** *Dashboard*: Develop a user-friendly dashboard around the library (RQ1, RQ3). In the dashboard, users compute and analyze results like the states and try different pipelines and configurations with the help of an LLM-based assistant, the Copilot.
- RG6** *Evaluation*: Evaluate the robustness and flexibility of the framework against different processes, event logs and timespans (RQ2, RQ5)
- RG7** *Limitations*: Reveal the limitations of the approach and potential misleading results especially when using the Copilot (RQ5)

1.5 Contributions

This thesis has six main contributions. Each contribution lists its research goals and chapters in parentheses.

- *Terminology*: A terminology defining and clearly separating drift types (what changes), drift modes (how it changes over time), and drift perspectives (which perspective the drift type belongs to) (RG1).
- *Rheon*: (RG2, Chapter 4). A library that generates synthetic event logs and injects nine drift types from all three perspectives, each suddenly or gradually. For each generated log, Rheon creates a metadata file comprehensively describing how the log was generated and a list of drifts together with their drift points, types and metadata.
- *State-based framework*: A framework that computes the intra-case, resource and inter-case states (RG3, Chapter 5). The framework consists of four steps: feature extraction, dimensionality reduction with PCA, clustering into states, and drift signals. The drift signals compare state distributions of calendar windows with divergences, or state vectors with similarity measures.
- *kairo*: A Python library that implements the framework (RG4, Section 6.1). With the help of its user-friendly documentation, users create a pipeline configuration in kairo and run it. The library supports various features addressing different drift types, various scaling techniques, PCA, and SOM, k-means and DBSCAN for state clustering. It also computes state frequency distributions for different window sizes, offers multiple divergences and similarity measures, and provides various plotting options for the different pipeline steps.
- *Dashboard*: A dashboard built around kairo (RG5, Section 6.2). In a user-friendly User Interface (UI), users can upload an event log, run the pipeline steps for each perspective and investigate the states in detail. With the LLM-based Copilot, users ask questions, gain insights about the results and request further steps.
- *Evaluation*: An evaluation on synthetic Rheon logs with different drift combinations from all perspectives and on the five real event logs of Padella and Vinci [21] (RG6, RG7, Chapters 7 and 8). The evaluation encompasses comprehensive comparisons between SOM, k-means and DBSCAN, and between different configuration parameters like scaling techniques, window sizes and feature selections.

1.6 Thesis Structure

The remainder of the thesis is structured as follows. Chapter 2 reviews the literature on concept drift detection and state-based process mining approaches and places this thesis within the literature. Chapter 3 introduces the notation for event logs, foundations of

concept drifts, PCA and clustering. Chapter 4 presents Rheon, which generates the synthetic event logs for the evaluation.

Chapter 5 defines intra-case, resource and inter-case states and answers RQ1. Each of its sections covers one step of the framework, i.e., feature extraction, dimensionality reduction, clustering into states, and drift signals, for all three perspectives. Chapter 6 describes the implementation of kairo and the dashboard tool.

Chapter 7 answers RQ2 to RQ5 using the Rheon logs and the benchmark logs of Padella and Vinci [21]. Chapter 8 discusses the design decisions and limitations of the framework. Chapter 9 summarizes the results and discusses future directions and potential extensions of the topic.

2 Related Work

This chapter gives an overview of the literature on different concept drift detection approaches, state-based process mining and positions the thesis in the literature. Section 2.1 presents detectors that focus on detecting control-flow drifts, and Section 2.2 focuses on the works on detectors beyond control-flow. Section 2.3 covers clustering-based detection approaches. Section 2.4 describes the role of states in process mining and reviews the state-based literature. In Section 2.5 we focus on publicly available event logs with known drifts as test data for the evaluation. Finally, Section 2.6 positions this thesis in the literature by explaining similarities and differences of our framework.

2.1 Drift Detection in Control-Flow

The most common strategy is a statistical test that compares the behavior of neighboring windows. In the survey of Sato et al. [23] 11 out of 36 papers use a statistical test approach. Bose et al. [8] apply statistical tests on follows relationships features between activities. The tool ProDrift developed by Maaradji et al. [19] uses a χ^2 test on the distribution of partially ordered runs, changes the window size adaptively and detects gradual drifts alongside sudden drifts. ProDrift receives higher F1-scores and detects drifts earlier than the approach of Bose et al. [8].

Other detectors represent the control-flow differently. In Tsinghua Process Concept Drift Detection (TPCDD) Zheng et al. [27] derive candidate drift points from a matrix of traces and relations and cluster them together using Density-Based Spatial Clustering of Applications with Noise (DBSCAN). With an Error Tolerance (ET) of 10 traces around the actual drift point, TPCDD achieves an F1-score of 1.0, whereas ProDrift needs an ET of about 100 traces for a comparable score on the same test data. CV4CDD-4D [17] learns to detect concept drifts using event logs with known sudden, gradual, incremental, and recurring drifts in an image of window similarities. CV4CDD-4D achieves an F1-score of 0.86 and finds the drift points more granular than the other approaches evaluated. It is also more robust against noise.

Beyond drift detection, Kraus and van der Aa [18] distinguish drift localization (what changed) from drift characterization (how it changed over time) and address the latter. In our terms, these correspond to drift types and drift modes.

All the detectors consider only the control-flow and therefore just the intra-case perspective of this thesis. Drifts that only change the waiting times, such as the one in Figure 1.1, stay hidden.

2.2 Drift Detection Beyond Control-Flow

Several detectors cover further perspectives. Brockhoff et al. [9] compare the trace distribution of two windows using the Earth Mover’s Distance (EMD) with regard to processing and sojourn times.

Three detectors detect drift points using the Pruned Exact Linear Time (PELT) algorithm on time series of features. Adams et al. [3] use control-flow, performance, data, resource, object features and test pairs of detected drifts for Granger causality. Klijn et al. [15] use features on resources and control-flow using an Event Knowledge Graph (EKG). CVDrift by Khraiweh et al. [14] chooses the window size of each time series from its Coefficient of Variation (CV), with series for control-flow, time, and resources. On event logs with activity duration drifts CVDrift achieves F1-scores from 0.77 to 0.97.

PELT-based detectors observe a drift in many features but capture the inter-case perspective only through individual features such as arrival time or processing time.

2.3 Clustering-Based Drift Detection

Methodologically, the clustering-based detectors are the most similar to this thesis. Clustering-based detectors observe how the composition of clusters changes over time and newly emerging or vanishing clusters are a drift signal (Sato et al. [23]). Concept-Drift in Event Stream Framework (CDESf) developed by Barbon Junior et al. [5] clusters cases of an event stream using DBSCAN with respect to distances in control-flow and time. CDESf signals a drift when outlier cases become dense enough to form a new cluster. Version Clustering (Abarca Zúñiga et al. [1]) clusters windows according to their directly-follows frequencies into process versions using Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN). It signals drift points at the transitions between these versions. Using a tolerance of 5% of the cases, Version Clustering achieves F1-scores of 0.84 and 0.96 on two benchmarks.

Both approaches apply clustering in a feature space describing mostly the control-flow. Version Clustering (Abarca Zúñiga et al. [1]) applies clustering on full windows similar to resource and inter-case perspectives of this thesis, but in our thesis the states do not represent a process version.

2.4 States in Process Mining

Classically a state describes a running case for making predictions. van der Aalst [25] predicts remaining time of cases using annotated transition systems with states (Figure 2.1). Senderovich et al. [24] encode running cases for prediction in a state space from intra-case and inter-case dependencies.

Berti et al. [6] learn execution states of objects using sliding windows, Principal Component Analysis (PCA) and a Self-Organizing Map (SOM). In this thesis, we adapt the approach to cases and extend it to resource and inter-case perspectives beyond the

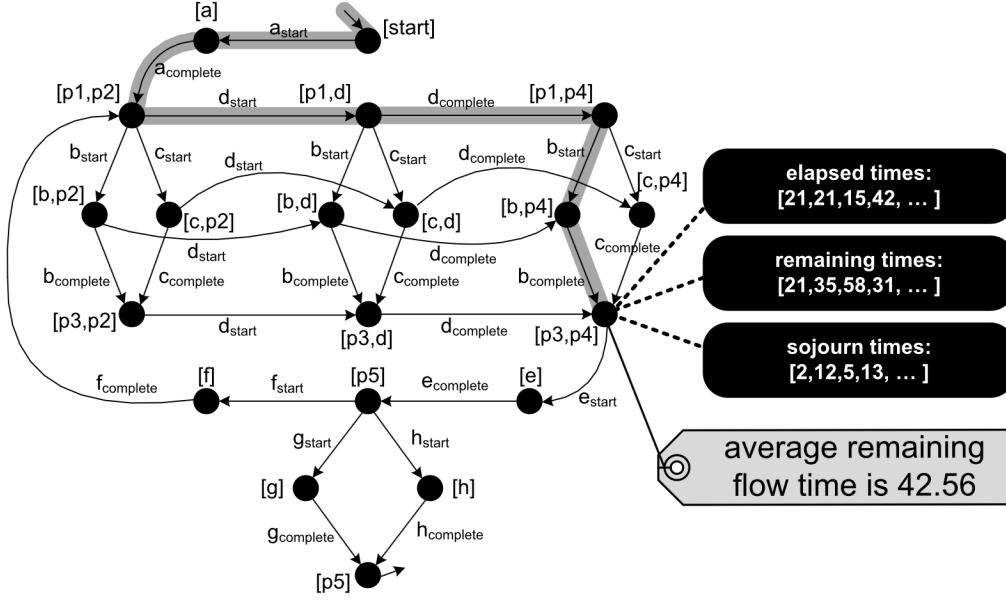


Figure 2.1: Transition system annotated with time measurements, taken from van der Aalst [25]. Each node is a state of a running case and each arc is the start or completion of an activity. Each state collects the elapsed, remaining, and sojourn times of all past cases that visited it, as shown for state [p3,p4]. The gray path is a running case that reaches [p3,p4]. Its predicted remaining flow time is 42.56, the average remaining time observed in this state.

intra-case perspective using calendar windows and feature extraction for concept drift detection.

Kraus et al. [16] detect steady states from Key Performance Indicators (KPIs) for each time window like the number of active cases. A fluctuating arrival rate is not seen as a concept drift by the paper whereas for this thesis it is a concept drift in the inter-case perspective.

2.5 Event Logs with Known Drifts

Adams et al. [2] compare eight detectors only on synthetic event logs with 132 control-flow drifts. None of the detectors performs well on all six evaluation goals, such as accuracy, latency, and robustness to noise. This kind of synthetic logs can be generated by the Concept Drift Log Generator (CDLG) [12]: It plays out process trees and injects drifts in the control-flow in four modes. The benchmark by Padella and Vinci [21], by contrast, has five real event logs, but with drifts only in control-flow and activity durations. None of these event logs has drifts in the resource perspective or arrival rates. Rheon follows the model of CDLG but adds multiple drift types from all three

perspectives supporting sudden and gradual drifts (Chapter 4).

2.6 Positioning of This Thesis

Three gaps motivate the approach of this thesis:

- *Perspectives*: Most detectors detect only control-flow drifts. Detectors for further perspectives recognize drift points in individual features or entire trace distributions but do not describe the process situation compactly.
- *States*: Clustering-based detectors mainly cluster control-flow behavior. The state-based works in Section 2.4 use states for prediction, object analysis, or steady-state detection, but not for concept drift detection.
- *Test data*: None of the aforementioned works and tools comprise resource drifts or arrival rate drifts.

This thesis addresses all three gaps. Our states describe a process in all three perspectives separately (research question RQ1), and we evaluate if their changes signal drifts (RQ2). Rheon delivers test data with injected drifts in all three perspectives (research goal RG2).

3 Preliminaries

In this chapter we introduce the notation and definitions that will be used in the next chapters. Section 3.1 formally defines event logs and traces, introduces process trees and defines calendar windows. Section 3.2 formalizes concept drifts with drift points and drift modes. Section 3.3 describes scaling of features and Principal Component Analysis (PCA). Section 3.4 presents the three clustering algorithms k-means, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Self-Organizing Maps (SOMs) used in our thesis.

3.1 Foundations

An event log captures how a process was executed. In the traditional case-centric approach, each event in an event log belongs to exactly one case, relates to exactly one activity and has a timestamp. Case identifier, activity and timestamp are traditionally the required columns in event logs for process mining. Additional attributes like resources, durations, event attributes as well as case attributes allow additional analysis. In Section 3.1.1, we introduce mathematical notation. Section 3.1.2 defines event logs, cases and traces, Section 3.1.3 introduces process trees, and Section 3.1.4 defines calendar windows.

3.1.1 Mathematical Preliminaries

Sets:

- A finite set $X = \{a_1, a_2, \dots, a_n\}$ is a finite collection of objects.
- The power set $\mathcal{P}(X)$ of X denotes the set of all subsets of X .
- $\mathbb{N}$ is the set of natural numbers, $\mathbb{Z}$ is the set of integers, $\mathbb{R}$ the set of real numbers and $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$ is the set of natural numbers including 0.

Functions:

- A function f from a set X to a set Y maps each element of its domain $\text{dom}(f) \subseteq X$ to exactly one element of Y .
- $X \rightharpoonup Y$ is the set of all functions from X to Y , which we call partial functions.
- $X \rightarrow Y$ is the set of all total functions from X to Y , i.e., of all $f \in X \rightharpoonup Y$ with $\text{dom}(f) = X$.

3 Preliminaries

- For a set Q , $f|_Q$ is the restriction of f to Q with $\text{dom}(f|_Q) = \text{dom}(f) \cap Q$ and $f|_Q(x) = f(x)$ for all $x \in \text{dom}(f|_Q)$.

Multisets:

- A multiset is a set where an element can occur multiple times.
- $\mathcal{B}(A) = A \rightarrow \mathbb{N}_0$ is the set of all multisets over a set A .
- For $X \in \mathcal{B}(A)$ and $a \in A$, $X(a)$ is the multiplicity of a in X .
- We write multisets as $X = [x^3, y^2, z]$, meaning $X(x) = 3$, $X(y) = 2$ and $X(z) = 1$.
- $[\]$ is the empty multiset, and $|X| = \sum_{a \in A} X(a)$ is the size of a finite multiset X .

Sequences:

- A sequence is an ordered collection of objects in which an object can occur multiple times.
- X^* is the set of all finite sequences over a set X , and $\langle \rangle$ is the empty sequence.
- A sequence $\sigma = \langle a_1, \dots, a_n \rangle \in X^*$ has the length $|\sigma| = n$, and $\sigma(i) = a_i$ is its i -th element for $1 \leq i \leq n$.
- For $0 \leq k \leq n$, $\text{prefix}_k(\sigma) = \langle a_1, \dots, a_k \rangle$ is the prefix of length k .
- $\partial_{\text{set}}(\sigma) = \{a_1, \dots, a_n\}$ turns σ into a set.
- $\partial_{\text{multiset}}(\sigma) = [a_1, \dots, a_n]$ turns σ into a multiset, e.g., $\partial_{\text{multiset}}(\langle a, b, b \rangle) = [a, b^2]$.
- $df(\sigma) = [(a_i, a_{i+1}) \mid 1 \leq i < n] \in \mathcal{B}(X \times X)$ is the multiset of directly-follows pairs of σ , e.g., $df(\langle a, b, b, c \rangle) = [(a, b), (b, b), (b, c)]$.

3.1.2 Event Logs, Cases and Traces

We define event logs as a set of events using an attribute function. In this notation, each event is uniquely identifiable even if two events have the same attribute values.

Universes:

- $\mathcal{U}_{ev}$ is the universe of events.
- $\mathcal{U}_{case}$ is the universe of case identifiers, $\mathcal{U}_{act}$ is the universe of activities and $\mathcal{U}_{res}$ is the universe of resources.
- $\mathcal{U}_{time}$ is the universe of timestamps.
- $\mathcal{U}_{att}$ is the universe of attribute names with $\{case, act, time, res\} \subseteq \mathcal{U}_{att}$.
- $\mathcal{U}_{val}$ is the universe of attribute values with $\mathcal{U}_{case} \cup \mathcal{U}_{act} \cup \mathcal{U}_{time} \cup \mathcal{U}_{res} \subseteq \mathcal{U}_{val}$.

- $\mathcal{U}_{map} = \mathcal{U}_{att} \rightarrow \mathcal{U}_{val}$ is the universe of attribute value mappings.

Definition 3.1.1 (Event Log). An event log is a tuple $L = (E, \pi, \preceq)$ with

- a finite set of events $E \subseteq \mathcal{U}_{ev}$,
- an attribute function $\pi \in E \rightarrow \mathcal{U}_{map}$ such that for each event $e \in E$:
 - $\{case, act, time\} \subseteq \text{dom}(\pi(e))$ with $\pi(e)(case) \in \mathcal{U}_{case}$, $\pi(e)(act) \in \mathcal{U}_{act}$ and $\pi(e)(time) \in \mathcal{U}_{time}$,
 - $res \in \text{dom}(\pi(e)) \Rightarrow \pi(e)(res) \in \mathcal{U}_{res}$,
- a total order $\preceq$ on E that respects the timestamps, i.e., $e \preceq e'$ for all $e, e' \in E$ with $\pi(e)(time) < \pi(e')(time)$.

Furthermore:

- For $e \in E$ and $n \in \text{dom}(\pi(e))$, $\pi_n(e) = \pi(e)(n)$ is the value of attribute n of e , e.g., $\pi_{act}(e)$ is the activity of e .
- An attribute name $n \in \mathcal{U}_{att} \setminus \{case, act, time, res\}$ is a case attribute of L if all events of a case have the same value for n , i.e., $\pi(e)|_{\{n\}} = \pi(e')|_{\{n\}}$ for all $e, e' \in E$ with $\pi_{case}(e) = \pi_{case}(e')$.

Definition 3.1.2 (Case, Trace and Prefix). Let $L = (E, \pi, \preceq)$ be an event log.

- $C_L = \{\pi_{case}(e) \mid e \in E\}$ is the set of cases of L .
- For a case $c \in C_L$, $E_L(c) = \{e \in E \mid \pi_{case}(e) = c\}$ is the set of its events.
- Let $e_1, \dots, e_n$ be the events of a case $c \in C_L$ in the order $\preceq$, i.e., $\{e_1, \dots, e_n\} = E_L(c)$, $n = |E_L(c)|$ and $e_i \preceq e_{i+1}$ for $1 \leq i < n$. Then
 - $trace_L(c) = \langle \pi_{act}(e_1), \dots, \pi_{act}(e_n) \rangle \in \mathcal{U}_{act}^*$ is the trace of c ,
 - $start_L(c) = \pi_{time}(e_1)$ is the start of c ,
 - $end_L(c) = \pi_{time}(e_n)$ is the end of c and
 - $prefix_L(e_k) = prefix_k(trace_L(c))$ is the prefix of the event e_k for $1 \leq k \leq n$.

Example 3.1.3. Table 3.1 shows the event log $L_1 = (E, \pi, \preceq)$ with the events $E = \{e_1, \dots, e_{11}\}$.

- $\mathcal{U}_{ev} = E = \{e_1, \dots, e_{11}\}$.
- $\mathcal{U}_{case} = \{1, 2, 3\}$, $\mathcal{U}_{act} = \{a, b, c, d\}$ and $\mathcal{U}_{res} = \{\text{Anna, Ben, Clara}\}$.
- $\mathcal{U}_{att} = \{case, act, time, res, duration, amount\}$.
- $\mathcal{U}_{val} = \mathbb{R} \cup \{a, b, c, d, \text{Anna, Ben, Clara}\} \cup \mathcal{U}_{time} \supseteq \mathcal{U}_{case} \cup \mathcal{U}_{act} \cup \mathcal{U}_{time} \cup \mathcal{U}_{res}$. The case IDs, durations and amounts are real numbers and therefore also in $\mathcal{U}_{val}$.

3 Preliminaries

Table 3.1: Event log L_1 with three cases and eleven events. Each row is an event with its attributes, and the second header row gives the attribute names. The activities are a, b, c and d . The timestamp is the completion time of an event, and the duration is its processing time in minutes. The amount in euros is a case attribute.

Event	Case	Activity	Timestamp	Resource	Duration	Amount
	<i>case</i>	<i>act</i>	<i>time</i>	<i>res</i>	<i>duration</i>	<i>amount</i>
e_1	1	a	2020-03-02 08:15	Anna	15	5000
e_2	1	b	2020-03-02 10:40	Ben	45	5000
e_3	1	c	2020-03-02 15:05	Clara	25	5000
e_4	1	d	2020-03-03 09:30	Anna	5	5000
e_5	2	a	2020-03-02 09:00	Anna	10	1200
e_6	2	c	2020-03-02 13:20	Clara	30	1200
e_7	2	d	2020-03-02 16:45	Ben	10	1200
e_8	3	a	2020-03-02 14:10	Anna	20	8000
e_9	3	b	2020-03-03 08:50	Ben	40	8000
e_{10}	3	b	2020-03-03 11:30	Ben	55	8000
e_{11}	3	c	2020-03-03 16:00	Clara	20	8000

- $\mathcal{U}_{map} = \mathcal{U}_{att} \rightarrow \mathcal{U}_{val}$ is the set of all partial functions from these six attribute names to values, e.g., $\{(act, b)\} \in \mathcal{U}_{map}$ and $\emptyset \in \mathcal{U}_{map}$.
- $\pi \in E \rightarrow \mathcal{U}_{map}$ maps each event to the attribute value mapping of its row in Table 3.1, e.g.,

$$\pi(e_9) = \{(case, 3), (act, b), (time, 2020-03-03 08:50), (res, Ben), (duration, 40), (amount, 8000)\}.$$

- For all attribute names $n \in \mathcal{U}_{att}$, the values of e_9 are $\pi_{case}(e_9) = 3$, $\pi_{act}(e_9) = b$, $\pi_{time}(e_9) = 2020-03-03 08:50$, $\pi_{res}(e_9) = Ben$, $\pi_{duration}(e_9) = 40$ and $\pi_{amount}(e_9) = 8000$.
- $amount$ is a case attribute, since all events of a case have the same amount, e.g., $\pi(e) \upharpoonright_{\{amount\}} = \{(amount, 8000)\}$ for the four events e of case 3.
- *Cases and traces:* L_1 has the cases $C_{L_1} = \{1, 2, 3\}$. Case 3 has the events $e_8 \preceq e_9 \preceq e_{10} \preceq e_{11}$ and thus the trace $trace_{L_1}(3) = \langle a, b, b, c \rangle$. It starts at 2020-03-02 14:10 and ends at 2020-03-03 16:00. The prefix of e_{10} is $\sigma = prefix_{L_1}(e_{10}) = \langle a, b, b \rangle$ with $\partial_{multiset}(\sigma) = [a, b^2]$ and $df(\sigma) = [(a, b), (b, b)]$.

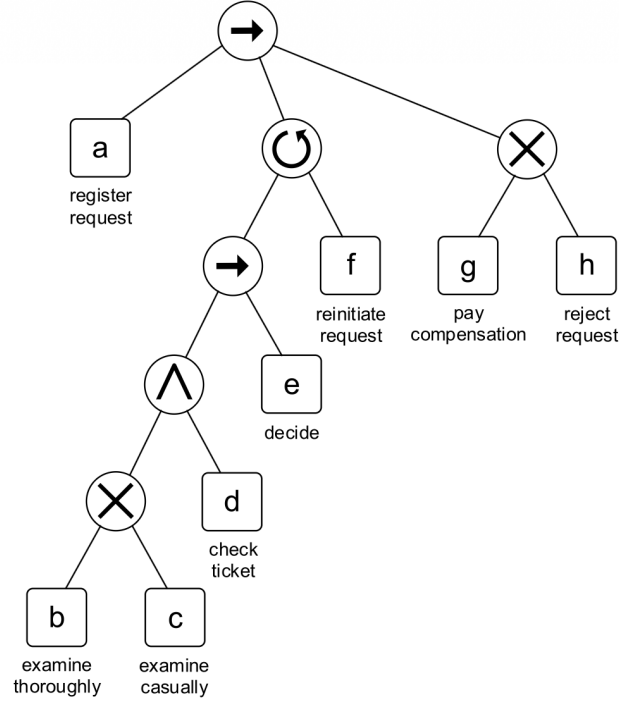


Figure 3.1: The process tree $\rightarrow(a, \circlearrowleft(\rightarrow(\wedge(\times(b, c), d), e), f), \times(g, h))$ showing different process tree operators. Taken from van der Aalst [25].

3.1.3 Process Trees

A process tree is a hierarchical process model whose leaves are activities and inner nodes operators (van der Aalst [25]). The children $Q_1, \dots, Q_m$ are themselves also process trees.

- *Sequential composition:* $\rightarrow(Q_1, \dots, Q_m)$ executes the children nodes step by step from left to right. E.g., $\rightarrow(a, b, c)$ produces only the trace $\langle a, b, c \rangle$.
- *Exclusive choice:* $\times(Q_1, \dots, Q_m)$ executes exactly one child. E.g., $\times(a, b, c)$ produces the traces $\langle a \rangle$, $\langle b \rangle$ and $\langle c \rangle$.
- *Parallel composition:* $\wedge(Q_1, \dots, Q_m)$ executes all children in any order. E.g., $\wedge(a, b, c)$ produces all six orderings of a , b and c .
- *Redo loop:* $\circlearrowleft(Q_1, \dots, Q_m)$ executes first the Do part Q_1 . Then, it stops or executes one of the Redo parts $Q_2, \dots, Q_m$ and again Q_1 . This can be repeated multiple times. E.g., $\circlearrowleft(a, b, c)$ produces the traces $\langle a \rangle$, $\langle a, b, a \rangle$, $\langle a, c, a \rangle$, $\langle a, b, a, c, a \rangle$ etc.

Figure 3.1 shows a nested process tree. It produces e.g., the traces $\langle a, b, d, e, g \rangle$ and $\langle a, d, c, e, f, b, d, e, h \rangle$.

3 Preliminaries

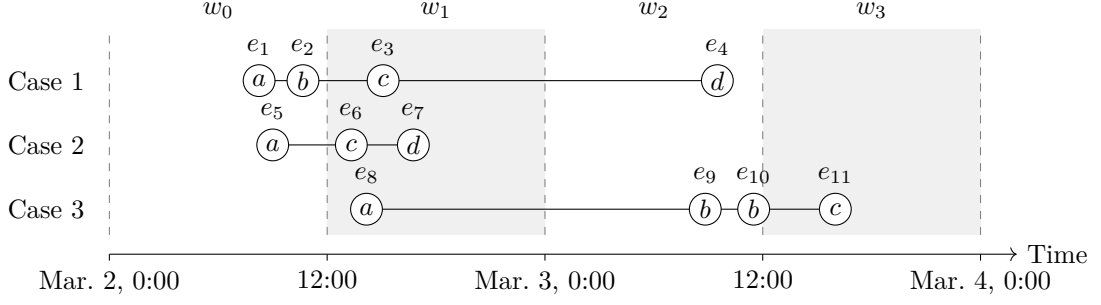


Figure 3.2: Event log L_1 from Table 3.1 on a time axis. Each row shows the events of a case in their order, and each circle is an event with its activity. Dashed lines separate the calendar windows w_0 to w_3 with the origin 2020-03-02 0:00 and the length $\Delta = 12$ hours. A case often spans several windows, and a window contains events of several cases.

3.1.4 Calendar Windows

We use calendar windows, i.e., time intervals of fixed length that start at calendar boundaries such as midnight. Thus, the window boundaries only depend on the origin and the window length, and not on the first event of the event log. To compute with timestamps, we treat them as real numbers, e.g., the seconds since a fixed point in time.

Definition 3.1.4 (Calendar Window). Let $o \in \mathcal{U}_{time}$ be an origin and $\Delta \in \mathbb{R}$ with $\Delta > 0$ a window length.

- For $k \in \mathbb{Z}$, $w_k = [o + k\Delta, o + (k + 1)\Delta)$ is the k -th calendar window.
- A timestamp $t \in \mathcal{U}_{time}$ lies in the window w_k if $t \in w_k$, i.e., $k = \lfloor (t - o)/\Delta \rfloor$.

Example 3.1.5. Figure 3.2 shows L_1 from Example 3.1.3 with the window length $\Delta = 12$ hours and the origin $o = 2020-03-02$ 0:00.

- For $k = 1$, the window is $w_1 = [2020-03-02$ 12:00, 2020-03-03 0:00).
- The timestamp $t = 2020-03-02$ 15:05 of e_3 lies in w_1 .
- The timestamps of e_3 , e_6 , e_7 and e_8 lie in w_1 , so w_1 contains events of all three cases.
- A case often spans several windows, e.g., the timestamps of case 3 lie in w_1 , w_2 and w_3 .

3.2 Concept Drift

In this section, we formalize concept drifts. The building block of a concept drift is the process change, which replaces one process version by another.

A concept drift is a situation in which a process changes. According to Kraus and van der Aa [18], an event log with a drift then contains data from different process versions. They describe a process version by the process model that governs its execution. We use a broader notion. Besides the control-flow, a process version also determines which resources execute which activities, how long activities take and how long a case waits between two events. It also determines how many cases arrive and which values their case attributes take. The drift type describes in which of these properties two versions differ (Section 1.1).

Definition 3.2.1 (Process Change). Let V be a set of process versions. A process change is a tuple $(v, v', t_s, t_e) \in V \times V \times \mathcal{U}_{time} \times \mathcal{U}_{time}$ with $v \neq v'$ and $t_s \leq t_e$ such that

- before t_s , the process behaves according to version v ,
- from t_e on, the process behaves according to version v' and
- between t_s and t_e , the process changes gradually from v to v' .

t_s and t_e are the drift points of the change. The change is sudden if $t_s = t_e$, and gradual otherwise.

In a gradual change, both versions coexist for some time, and the old version is discontinued gradually. We assume that the share of behavior that follows v' grows linearly between t_s and t_e .

Definition 3.2.2 (Concept Drift, Drift Mode). Let V be a set of process versions. A concept drift is a sequence

$$\delta = \langle (v_1, v_2, s_1, t_1), (v_2, v_3, s_2, t_2), \dots, (v_m, v_{m+1}, s_m, t_m) \rangle$$

of $m \geq 1$ process changes with $t_i \leq s_{i+1}$ for $1 \leq i < m$. The drift points of δ are the drift points of its changes. The drift mode of δ is

- *sudden* if $m = 1$ and $s_1 = t_1$,
- *gradual* if $m = 1$ and $s_1 < t_1$,
- *recurring* if $m \geq 2$ and a version occurs more than once, i.e., $v_i = v_j$ for some $1 \leq i < j \leq m + 1$,
- *incremental* if $m \geq 2$, the versions $v_1, \dots, v_{m+1}$ are pairwise different and each change alters the process only slightly, where all changes follow the same initiative.

The four drift modes originate from the taxonomy of Bose et al. [8], and Kraus and van der Aa [18] call them drift types. We call them drift modes instead and reserve the term drift type for what changes. In this thesis we will focus on sudden and gradual drifts. Figure 3.3 shows the four drift modes.

Example 3.2.3. The following drift illustrates Definitions 3.2.1 and 3.2.2.

3 Preliminaries

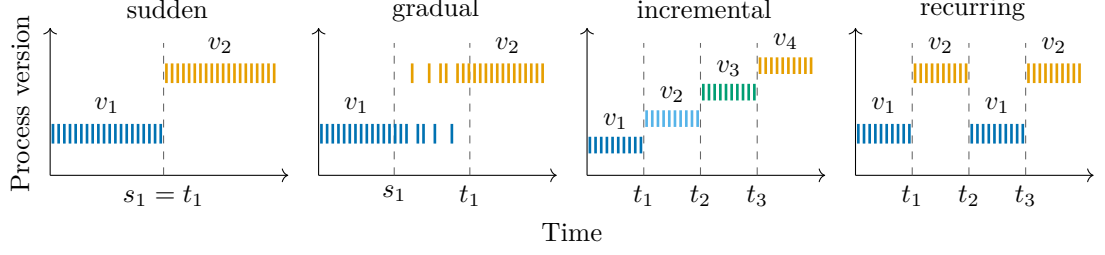


Figure 3.3: The four drift modes. Each tick is a case, and its height and color show the process version it follows. Dashed lines mark the drift points. A sudden change replaces v_1 by v_2 at one point in time. In a gradual change, more and more cases follow v_2 between s_1 and t_1 . An incremental drift leads from v_1 to v_4 in small steps, and a recurring drift returns to earlier versions. The changes of the incremental and the recurring drift are sudden here, i.e., $s_i = t_i$.

- In the Rheon log of Figure 1.1, the mean waiting time between two consecutive events of a case rises from 14.2 to 45 minutes on April 19, 2020. This is a sudden drift $\langle (v_1, v_2, t_1, t_1) \rangle$ with $t_1 = \text{April 19, 2020}$.
- The versions v_1 and v_2 only differ in the waiting time, so the drift type is a pure waiting time drift.

3.3 Scaling and Principal Component Analysis

Each event or calendar window is described by numerical features, which we write as vectors.

Feature vectors:

- $\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$ is the feature vector of an event or a calendar window with its features $x_1, \dots, x_n$.
- A feature matrix $X \in \mathbb{R}^{m \times n}$ contains m feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_m$ as its rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$, and $x_{j,i}$ is the value of the i -th feature in the j -th vector.
- $\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2}$ is the Euclidean norm of $\mathbf{x}$, and $\|\mathbf{x} - \mathbf{y}\|$ is the Euclidean distance of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$.
- Further distances of $\mathbf{x}$ and $\mathbf{y}$ are the Manhattan distance $\sum_{i=1}^n |x_i - y_i|$, the Chebyshev distance $\max_{1 \leq i \leq n} |x_i - y_i|$ and, for $\mathbf{x}, \mathbf{y} \neq \mathbf{0}$, the cosine distance $1 - \mathbf{x}^\top \mathbf{y} / (\|\mathbf{x}\| \|\mathbf{y}\|)$.

Features may have different units and ranges, e.g., binary, one-hot, minutes, counts and shares. If their variances differ strongly, the variances determine the directions of

3.3 Scaling and Principal Component Analysis

the principal components more than the correlations. Therefore, each feature is usually brought to mean 0 and variance 1 before PCA. An alternative is the min-max normalization to the interval $[0, 1]$.

Definition 3.3.1 (Scaling). Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with $m \geq 2$. For the i -th feature, let

- $\mu_i = \frac{1}{m} \sum_{j=1}^m x_{j,i}$ be its mean,
- $s_i = \sqrt{\frac{1}{m-1} \sum_{j=1}^m (x_{j,i} - \mu_i)^2}$ its standard deviation and
- $\underline{x}_i = \min_{1 \leq j \leq m} x_{j,i}$ and $\bar{x}_i = \max_{1 \leq j \leq m} x_{j,i}$ its smallest and largest value.

The scaled matrix $X' \in \mathbb{R}^{m \times n}$ has the entries

- $x'_{j,i} = (x_{j,i} - \mu_i)/s_i$ for the *z-score standardization*, after which each non-constant feature has mean 0 and standard deviation 1, and
- $x'_{j,i} = (x_{j,i} - \underline{x}_i)/(\bar{x}_i - \underline{x}_i)$ for the *min-max normalization*, after which all values lie in $[0, 1]$.

For a constant feature, i.e., $s_i = 0$ or $\underline{x}_i = \bar{x}_i$, we set $x'_{j,i} = 0$ for $1 \leq j \leq m$.

PCA projects vectors onto a few directions in which they spread the most. It thus compresses many, often correlated features into a few components.

Definition 3.3.2 (Principal Component Analysis). Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$.

- The mean vector $\boldsymbol{\mu} \in \mathbb{R}^n$ and the covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$ of X are

$$\boldsymbol{\mu} = \frac{1}{m} \sum_{j=1}^m \mathbf{x}_j \quad \text{and} \quad \Sigma = \frac{1}{m} \sum_{j=1}^m (\mathbf{x}_j - \boldsymbol{\mu})(\mathbf{x}_j - \boldsymbol{\mu})^\top.$$

- Let $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ be the eigenvalues of Σ and $\mathbf{v}_1, \dots, \mathbf{v}_n$ corresponding orthonormal eigenvectors, i.e., $\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$. Then $\mathbf{v}_i$ is the i -th principal component.
- For $1 \leq d \leq n$, let $W_d = [\mathbf{v}_1, \dots, \mathbf{v}_d] \in \mathbb{R}^{n \times d}$. The projection of a vector $\mathbf{x} \in \mathbb{R}^n$ onto the first d principal components is $\mathbf{z} = W_d^\top (\mathbf{x} - \boldsymbol{\mu}) \in \mathbb{R}^d$, and its reconstruction is $\hat{\mathbf{x}} = \boldsymbol{\mu} + W_d \mathbf{z}$.
- The explained variance ratio of the first d principal components and the reconstruction error are

$$\text{evr}_X(d) = \frac{\sum_{i=1}^d \lambda_i}{\sum_{i=1}^n \lambda_i} \quad \text{and} \quad \text{err}_X(d) = \frac{1}{m} \sum_{j=1}^m \|\mathbf{x}_j - \hat{\mathbf{x}}_j\|^2,$$

where $\hat{\mathbf{x}}_j$ is the reconstruction of $\mathbf{x}_j$, and $\text{evr}_X(d)$ is only defined if $\sum_{i=1}^n \lambda_i > 0$.

3 Preliminaries

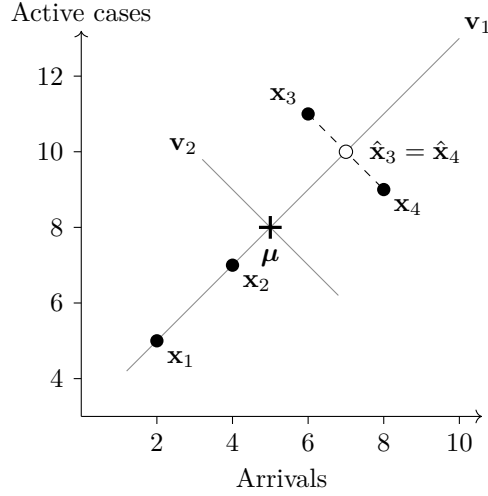


Figure 3.4: PCA for the four feature vectors of Example 3.3.3. The gray lines are the principal components through the mean μ (cross). The variance along $\mathbf{v}_1$ is $\lambda_1 = 9$, along $\mathbf{v}_2$ only $\lambda_2 = 1$. Thus, the projection onto $\mathbf{v}_1$ keeps 90% of the variance. The reconstructions of $\mathbf{x}_1$ and $\mathbf{x}_2$ are exact. $\mathbf{x}_3$ and $\mathbf{x}_4$ are both reconstructed as $(7, 10)^\top$ (empty circle), and the dashed lines are their reconstruction errors.

Properties:

- The first principal component is the direction in which the projected vectors have the largest variance. Each further component maximizes the variance among all directions that are orthogonal to the previous ones, and the variance along $\mathbf{v}_i$ is λ_i (Alpaydin [4]).
- Among all orthogonal linear projections onto d dimensions, PCA has the smallest reconstruction error (Alpaydin [4]).
- We choose d at the elbow of $err_X(d)$ over d , i.e., where a further component hardly lowers the reconstruction error.

Example 3.3.3. Four calendar windows are described by their number of arrivals and their number of active cases.

- The feature vectors $\mathbf{x}_1 = (2, 5)^\top$, $\mathbf{x}_2 = (4, 7)^\top$, $\mathbf{x}_3 = (6, 11)^\top$ and $\mathbf{x}_4 = (8, 9)^\top$ have the mean vector and the covariance matrix

$$\mu = \begin{pmatrix} 5 \\ 8 \end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}.$$

- Both features have the same variance, so scaling does not change the principal components.

- Σ has the eigenvalues $\lambda_1 = 9$ and $\lambda_2 = 1$ with the eigenvectors $\mathbf{v}_1 = (1, 1)^\top / \sqrt{2}$ and $\mathbf{v}_2 = (-1, 1)^\top / \sqrt{2}$.
- The first principal component explains $\text{evr}_X(1) = 9/10$ of the variance, since windows with many arrivals also have many active cases.
- The projections onto $\mathbf{v}_1$ are $-3\sqrt{2}$, $-\sqrt{2}$, $2\sqrt{2}$ and $2\sqrt{2}$.
- The reconstructions of $\mathbf{x}_1$ and $\mathbf{x}_2$ are exact. $\mathbf{x}_3$ and $\mathbf{x}_4$ are both reconstructed as $(7, 10)^\top$ with the squared error 2, so $\text{err}_X(1) = (0 + 0 + 2 + 2)/4 = 1 = \lambda_2$ (Figure 3.4).

3.4 Clustering

A clustering algorithm groups vectors such that vectors in the same group are similar to each other. Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$. A clustering of X is a function $cl \in \{1, \dots, m\} \rightarrow K$ that assigns each vector a cluster from a set K . The following three algorithms form clusters in different ways:

- *k-means* forms a given number of clusters around centers (Section 3.4.1).
- A *self-organizing map* arranges prototypes on a grid whose cells are the clusters (Section 3.4.2).
- *DBSCAN* merges dense regions into clusters and marks the vectors outside them as noise (Section 3.4.3).

Figure 3.5 applies all three algorithms to the same vectors. Section 3.4.4 introduces metrics that evaluate the resulting clusterings.

3.4.1 k-Means

k-means searches k centers such that the squared distances of the vectors to their nearest center are as small as possible.

Definition 3.4.1 (k-Means). Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$ and $k \in \mathbb{N}$ with $k \leq m$. A k-means clustering of X consists of centers $\mathbf{c}_1, \dots, \mathbf{c}_k \in \mathbb{R}^n$ and a clustering $cl \in \{1, \dots, m\} \rightarrow \{1, \dots, k\}$ such that

- each vector belongs to its nearest center, i.e., $cl(j) \in \arg \min_{1 \leq l \leq k} \|\mathbf{x}_j - \mathbf{c}_l\|$ for $1 \leq j \leq m$, and
- each center is the mean of its vectors, i.e., for $1 \leq l \leq k$,

$$\mathbf{c}_l = \frac{1}{|J_l|} \sum_{j \in J_l} \mathbf{x}_j \quad \text{with} \quad J_l = \{j \mid cl(j) = l\} \neq \emptyset.$$

3 Preliminaries

k-means finds such a clustering by starting with k randomly chosen vectors as centers. It then establishes both conditions alternately until the assignment does not change anymore.

Properties:

- k-means needs the number k of clusters in advance.
- Outliers always belong to a cluster.
- The result depends on the initial centers.

3.4.2 Self-Organizing Maps

A SOM is a neural network whose neurons lie on a usually two-dimensional grid. The model goes back to Kohonen [13]. We call a neuron a cell of the grid. Each cell has a prototype in the feature space, and a vector belongs to the cell with the nearest prototype. During training, the cells compete for each vector, the winning cell pulls its neighbors along, and the prototypes adapt to the vector.

Definition 3.4.2 (Self-Organizing Map). Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$ and $G = \{1, \dots, p\} \times \{1, \dots, q\}$ a grid with $p, q \in \mathbb{N}$.

- A SOM assigns each cell $\mathbf{r} \in G$ a prototype $\mathbf{w}_{\mathbf{r}}(t) \in \mathbb{R}^n$ at time $t \in \mathbb{N}_0$.
- The Best-Matching Unit (BMU) of a vector $\mathbf{x} \in \mathbb{R}^n$ at time t is the cell $b_t(\mathbf{x}) = \arg \min_{\mathbf{r} \in G} \|\mathbf{x} - \mathbf{w}_{\mathbf{r}}(t)\|$.
- The initial prototypes $\mathbf{w}_{\mathbf{r}}(0)$ are, e.g., randomly chosen vectors of X .
- In each step $t = 0, 1, \dots, T - 1$, a vector $\mathbf{x}(t) \in \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ is chosen, and all prototypes move towards it:

$$\mathbf{w}_{\mathbf{r}}(t+1) = \mathbf{w}_{\mathbf{r}}(t) + \eta(t) h_{b_t(\mathbf{x}(t)), \mathbf{r}}(t) (\mathbf{x}(t) - \mathbf{w}_{\mathbf{r}}(t)) \quad \text{for all } \mathbf{r} \in G.$$

- The neighborhood function $h_{\mathbf{b}, \mathbf{r}}(t) = \exp(-\|\mathbf{b} - \mathbf{r}\|^2 / (2\sigma(t)^2))$ decreases with the grid distance of the cells $\mathbf{b}$ and $\mathbf{r}$.
- The learning rate $\eta(t) > 0$ and the neighborhood radius $\sigma(t) > 0$ decrease monotonically in t .
- After T steps, the SOM yields the clustering $cl(j) = b_T(\mathbf{x}_j)$ with $K = G$.

Properties:

- The neighborhood function also pulls the neighbors of the BMU towards the vector. Thus, neighboring cells get similar prototypes, and vectors that are close in the feature space fall into close cells.

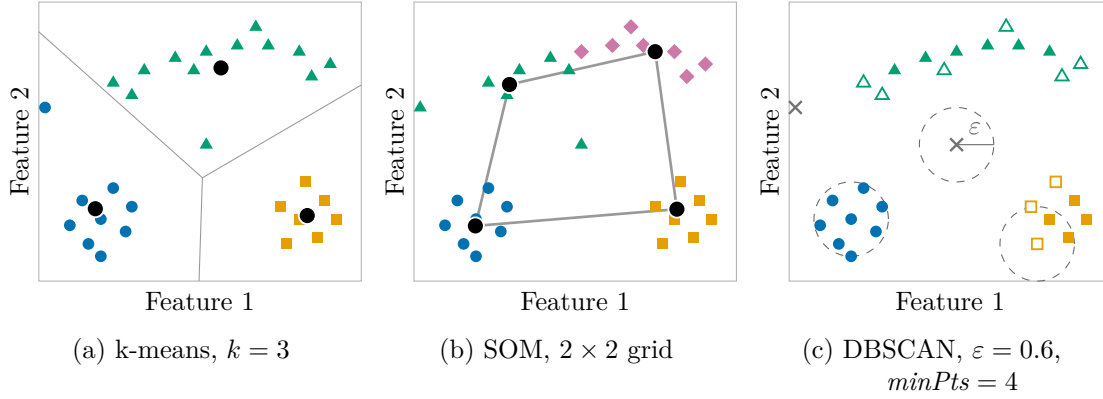


Figure 3.5: The three clustering algorithms of Example 3.4.4 on the same 29 vectors with three groups and two outliers. Color and shape of a symbol show its cluster, and black dots are the centers or prototypes. (a) k-means divides the plane into convex cells (gray lines) and also assigns the outliers to a cluster. (b) The gray line connects the prototypes of neighboring cells: (1, 1) bottom left, (1, 2) bottom right, (2, 2) top right and (2, 1) top left. The elongated group is split into the cells (2, 1) and (2, 2). (c) DBSCAN detects the outliers as noise (gray crosses). Filled symbols are core points and empty symbols border points. The dashed circles show the ε -neighborhood of a core point, a border point and an outlier.

- Towards the end of the training, $\sigma(t)$ is so small that almost only the BMU adapts.
- The SOM forms at most $p \cdot q$ clusters.
- As in k-means, every vector belongs to a cluster.
- Similarly to k-means, the grid size must be specified in advance.
- Instead of the Euclidean distance, the BMU can be determined with another distance, e.g., the Manhattan, Chebyshev or cosine distance. The update of the prototypes stays the same.

3.4.3 DBSCAN

DBSCAN forms clusters from regions in which vectors lie densely (Ester et al. [11]). A region is dense if at least minPts vectors lie within the radius ε of a vector. Vectors in sparse regions belong to no cluster and count as noise.

Definition 3.4.3 (DBSCAN). Let $X \in \mathbb{R}^{m \times n}$ be a feature matrix with the rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$, $I = \{1, \dots, m\}$, $\varepsilon \in \mathbb{R}$ with $\varepsilon > 0$ and $\text{minPts} \in \mathbb{N}$.

- $N_\varepsilon(j) = \{l \in I \mid \|\mathbf{x}_j - \mathbf{x}_l\| \leq \varepsilon\}$ is the ε -neighborhood of $j \in I$. It contains j itself.

3 Preliminaries

- $j \in I$ is a core point if $|N_\varepsilon(j)| \geq \text{minPts}$.
- $l \in I$ is directly density-reachable from $j \in I$ if j is a core point and $l \in N_\varepsilon(j)$. l is density-reachable from j if there are $i_1, \dots, i_r \in I$ with $i_1 = j$ and $i_r = l$ such that i_{h+1} is directly density-reachable from i_h for $1 \leq h < r$.
- A cluster is a non-empty set $C \subseteq I$ that is maximal and connected. Maximal means that $l \in C$ if $j \in C$ and l is density-reachable from j . Connected means that for all $j, l \in C$, there is an $i \in I$ from which both j and l are density-reachable.
- The points of a cluster that are not core points are border points. Points that belong to no cluster are noise.
- For the clusters $C_1, \dots, C_k$, the clustering is $cl(j) = l$ for $j \in C_l$ and $cl(j) = \text{noise}$ if j is noise. Thus, $K = \{1, \dots, k, \text{noise}\}$.

Properties:

- Unlike in k-means and the SOM, the number of clusters follows from the data.
- Any distance can replace the Euclidean distance in the ε -neighborhood [11], e.g., the Manhattan, Chebyshev or cosine distance.
- A common approach to choose ε is using the first valley area of the sorted k -dist graph with $k = \text{minPts}$. This graph shows the distance of each point to its k -th nearest neighbor in descending order.

Example 3.4.4. Figure 3.5 shows 29 vectors in $\mathbb{R}^2$: three groups, one of them elongated, and two outliers.

- k-means with $k = 3$ finds the three groups, but also assigns both outliers to a cluster.
- The SOM with a 2×2 grid has four cells for three groups. Therefore, it splits the elongated group into two neighboring cells.
- DBSCAN with $\varepsilon = 0.6$ and $\text{minPts} = 4$ finds exactly the three groups and detects both outliers as noise.

3.4.4 Clustering Evaluation

The true groups of the vectors are unknown. Thus, a clustering can only be evaluated with the vectors themselves, e.g., by how compact and how well separated its clusters are. We use five such metrics. The silhouette and the Caliński–Harabasz index evaluate the clusterings of all three algorithms. The quantization error and the occupancy entropy evaluate SOMs, and Density-Based Clustering Validation (DBCV) evaluates DBSCAN. For SOMs and DBSCAN, the silhouette, the quantization error and DBCV use the

same distance as the clustering. The Caliński–Harabasz index always uses the Euclidean distance.

Clusters:

- The clusters of a clustering cl of a feature matrix $X \in \mathbb{R}^{m \times n}$ are the non-empty sets $\{j \mid cl(j) = \kappa\}$ with $\kappa \in K \setminus \{\text{noise}\}$. We number them $C_1, \dots, C_k$.
- Noise belongs to no cluster, and $I' = C_1 \cup \dots \cup C_k$ contains the $m' = |I'|$ clustered vectors.

The silhouette compares the distance of a vector to its own cluster with its distance to the nearest other cluster [22]. The Caliński–Harabasz index compares the dispersion between the clusters with the dispersion within them [10].

Definition 3.4.5 (Silhouette). Let cl be a clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters and $m' > k$. For $j \in C_l$, let

$$a(j) = \frac{1}{|C_l| - 1} \sum_{i \in C_l \setminus \{j\}} \|\mathbf{x}_j - \mathbf{x}_i\| \quad \text{and} \quad b(j) = \min_{l' \neq l} \frac{1}{|C_{l'}|} \sum_{i \in C_{l'}} \|\mathbf{x}_j - \mathbf{x}_i\|$$

be the mean distances of $\mathbf{x}_j$ to the other vectors of its cluster and to the vectors of the nearest other cluster.

- The silhouette of j is

$$s(j) = \begin{cases} \frac{b(j) - a(j)}{\max\{a(j), b(j)\}} & \text{if } |C_l| > 1, \\ 0 & \text{if } |C_l| = 1. \end{cases}$$

- $sil(cl) = \frac{1}{m'} \sum_{j \in I'} s(j)$ is the silhouette of cl .

Definition 3.4.6 (Caliński–Harabasz Index). Let cl be a clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters and $m' > k$. Let $\boldsymbol{\mu}'$ be the mean of the clustered vectors $\mathbf{x}_j$ with $j \in I'$, and $\mathbf{c}_l$ the mean of the vectors of C_l .

- $D_b = \sum_{l=1}^k |C_l| \|\mathbf{c}_l - \boldsymbol{\mu}'\|^2$ is the dispersion between the clusters.
- $D_w = \sum_{l=1}^k \sum_{j \in C_l} \|\mathbf{x}_j - \mathbf{c}_l\|^2$ is the dispersion within the clusters.
- For $D_w > 0$, the Caliński–Harabasz index of cl is

$$ch(cl) = \frac{D_b/(k-1)}{D_w/(m'-k)}.$$

Properties:

- $s(j) > 0$ if $\mathbf{x}_j$ lies closer to its own cluster than to the nearest other one. A silhouette close to 1 shows that $\mathbf{x}_j$ lies well within its cluster, and one close to 0 that it lies between two clusters.

3 Preliminaries

- $-1 \leq sil(cl) \leq 1$, and Rousseeuw [22] uses the mean silhouette to choose the number of clusters.
- $ch(cl)$ is large if the clusters are compact and lie far apart. It has no upper bound and depends on the scale of the features. Thus, it only compares clusterings of the same vectors.

For SOMs, we also measure how closely the prototypes fit the vectors and how evenly the vectors spread over the grid (Berti et al. [6]).

Definition 3.4.7 (Quantization Error, Occupancy Entropy). Let cl be the clustering of a SOM after T steps (Definition 3.4.2) with the prototypes $\mathbf{w}_{\mathbf{r}} = \mathbf{w}_{\mathbf{r}}(T)$ on a grid G with $p \cdot q \geq 2$ cells.

- $qe(cl) = \frac{1}{m} \sum_{j=1}^m \|\mathbf{x}_j - \mathbf{w}_{cl(j)}\|$ is the quantization error, i.e., the mean distance of the vectors to the prototypes of their cells.
- $occ(\mathbf{r}) = |\{j \mid cl(j) = \mathbf{r}\}|/m$ is the occupancy of a cell $\mathbf{r} \in G$, i.e., the share of the vectors in $\mathbf{r}$.
- The occupancy entropy of cl is

$$ent(cl) = -\frac{1}{\log(p \cdot q)} \sum_{\mathbf{r} \in G, occ(\mathbf{r}) > 0} occ(\mathbf{r}) \log occ(\mathbf{r}).$$

Properties:

- A small quantization error means that the prototypes lie close to their vectors.
- The occupancy entropy is 0 if all vectors lie in one cell and 1 if all cells hold equally many vectors.

Moulavi et al. [20] designed DBCV to compare the density within the clusters with the density between them. In a good clustering, even the sparsest region inside a cluster is denser than the region that separates it from the other clusters.

Definition 3.4.8 (Density-Based Clustering Validation). Let cl be a DBSCAN clustering of a feature matrix $X \in \mathbb{R}^{m \times n}$ with $k \geq 2$ clusters.

- The density sparseness $dsc(C_l)$ is the largest gap that the vectors of C_l have to bridge to connect each other, i.e., the sparsest region inside C_l .
- The density separation $sep(C_l)$ is the smallest gap between C_l and another cluster, i.e., the densest region between them.
- The validity of C_l is

$$v(C_l) = \frac{sep(C_l) - dsc(C_l)}{\max\{sep(C_l), dsc(C_l)\}}.$$

- $dbcv(cl) = \sum_{l=1}^k \frac{|C_l|}{m} v(C_l)$ is the DBCV of cl , where m also counts the noise.

Properties:

- Both gaps are taken from a minimum spanning tree of each cluster, whose edge weights grow in sparse regions. The tree also captures the shape of the cluster, so DBCV evaluates clusters of arbitrary shape [20].
- $v(C_l) > 0$ if C_l is denser inside than towards the nearest other cluster. The DBCV lies between -1 and 1 [20].
- Noise lowers the DBCV, since it counts in m but belongs to no cluster [20].

Table 3.2: Metrics of the three clusterings in Figure 3.5.

Clustering	<i>sil</i>	<i>ch</i>	<i>qe</i>	<i>ent</i>	<i>dbcv</i>
k-means, $k = 3$	0.573	45.4	–	–	–
SOM, 2×2 grid	0.585	65.5	0.575	0.999	–
DBSCAN, $\varepsilon = 0.6$, $minPts = 4$	0.627	52.7	–	–	0.684

Example 3.4.9. Table 3.2 evaluates the clusterings of Example 3.4.4.

- DBSCAN has the highest silhouette, since it leaves out the two outliers. In k-means, the outlier (2.7, 2.2) only has the silhouette 0.125.
- The elongated group has the lowest mean silhouette in all three clusterings.
- The SOM has the highest Caliński–Harabasz index, although it splits the elongated group.
- The SOM uses its four cells almost evenly with 8, 7, 7 and 7 vectors, so its occupancy entropy is close to 1. Its quantization error is the mean distance of the vectors to the black dots in Figure 3.5b.
- DBSCAN has the DBCV 0.684. Since the two noise vectors count in $m = 29$, the DBCV is at most $27/29 \approx 0.931$ here.

4 Rheon

To evaluate whether a detector detects a drift, we need event logs with known drifts. Public event logs with known drifts are hard to find and mostly contain drifts in the control-flow perspective or timing attributes. The Concept Drift Log Generator (CDLG) developed by Grimm et al. [12] generates synthetic event logs with control-flow drifts and the authors mention multi-perspective drifts as future work.

We introduce Rheon in this chapter in order to fill this gap. Rheon is a Python library developed in this thesis that generates synthetic event logs with injected concept drifts. It allows users to configure the generation parameters comprehensively and to inject concept drifts from intra-case, resource and inter-case perspectives. The input comprises parameters of a base process model and a list of drifts. The output is an event log with activities, timestamps, resources, two case attributes and a metadata file that comprehensively describes the base process parameters as well as the injected drifts together with their drift types. Rheon supports nine drift types from the three perspectives and the two drift modes sudden and gradual. We will use Rheon logs in Chapter 7 to evaluate our state-based framework on known drifts.

Section 4.1 introduces the log schema of Rheon-generated logs and gives an example. Section 4.2 gives an overview of the generation of an event log using Rheon. Section 4.3 introduces the nine drift types supported by Rheon. In Section 4.4 we give examples of generated logs and figures showing the drifts visually. Section 4.5 discusses the limitations and future work of Rheon.

4.1 Generated Log Schema

Rheon generates and writes an event log according to Definition 3.1.1 as a file in the format eXtensible Event Stream (XES) or Comma-Separated Values (CSV). Table 4.1 shows the nine columns of the event log schema. Amount and region are the case attributes. Rheon sorts the events according to case identifiers and start timestamps. The first event of a case starts at the arrival time of the case. All other events start after the end of the previous event and a waiting time. For all generated event logs, Rheon also creates a metadata file that includes the generation parameters and the drifts of the log. Table 4.2 provides an example of a Rheon-generated log.

4.2 Generation of an Event Log

Similar to CDLG (Grimm et al. [12]), Rheon describes the control-flow through a process tree (Section 3.1.3) and generates traces with a play-out from a process tree. For the

Table 4.1: The columns of a Rheon log. The level states whether an attribute belongs to an event or to a case. The column Name gives the attribute name in Definition 3.1.1.

Column	Level	Name	Meaning
<code>event:id</code>	event	–	event identifier
<code>case:concept:name</code>	case	<i>case</i>	case identifier
<code>concept:name</code>	event	<i>act</i>	activity
<code>start_timestamp</code>	event	–	start of processing
<code>time:timestamp</code>	event	<i>time</i>	completion of processing
<code>event:duration_min</code>	event	<i>duration</i>	processing time
<code>org:resource</code>	event	<i>res</i>	assigned resource
<code>case:amount</code>	case	<i>amount</i>	case amount
<code>case:region</code>	case	<i>region</i>	case region

Table 4.2: The first six rows of the example log with a control-flow drift from Section 4.4. Identifiers are abbreviated, and timestamps are shown without year and seconds.

event: id	case: concept: name	concept: name	start_ timestamp	time: timestamp	event: duration_ min	org: resource	case: amount	case: region
evt_1	case_1	a	Jan 1, 00:00	Jan 1, 00:24	24.20	res_03	1,151.36	region_3
evt_2	case_1	b	Jan 1, 00:41	Jan 1, 01:10	29.48	res_01	1,151.36	region_3
evt_3	case_1	c	Jan 1, 01:48	Jan 1, 02:08	20.33	res_01	1,151.36	region_3
evt_4	case_1	d	Jan 1, 02:21	Jan 1, 02:37	16.27	res_04	1,151.36	region_3
evt_5	case_1	e	Jan 1, 02:38	Jan 1, 02:59	21.57	res_01	1,151.36	region_3
evt_6	case_2	a	Jan 1, 09:00	Jan 1, 09:03	3.10	res_03	768.65	region_3

generation and the play-out, Rheon uses the library PM4Py (Berti et al. [7]). Rheon currently provides a single function `rheon.generate_log`. Table 4.3 gives an overview of the parameters of the function. A fixed seed makes the random numbers reproducible, but the play-out of PM4Py for parallel branches can yield non-deterministic interleavings of activities.

Figure 4.1 gives a visual overview of the workflow of log generation.

1. *Build process trees and pools:* Rheon generates a random base process tree T_{base} with `num_activities` activities and the operator weights `tree_weights`. For the n control-flow drifts in `drifts`, it generates n further process trees $T_1, \dots, T_n$. T_i is the process tree after the i -th control-flow drift and differs from the process tree before it. Each process tree is thus valid in one period of the time horizon, and these periods are separated by the control-flow drifts. Then, the play-out of each process tree gives a pool of traces, i.e., many traces of this process version.

Table 4.3: The parameters of `rheon.generate_log` with their default values. Means of times are given in minutes and their variances in squared minutes.

Parameter	Default	Meaning
<code>drifts</code>	required	list of drifts to inject
<code>output_path</code>	required	path of the log
<code>log_name</code>	file name	name in the metadata and of the metadata file
<code>format</code>	"xes"	output format "xes" or "csv"
<code>num_traces</code>	1,000	approximate number of cases
<code>num_activities</code>	10	activities of the base process tree
<code>num_resources</code>	8	size of the resource pool
<code>num_regions</code>	4	number of regions
<code>tree_weights</code>	0.6, 0.25, 0.1, 0.05	weights of sequence, choice, parallelism and loop
<code>start_date, end_date</code>	Jan 1, 2020, Dec 31, 2020	time horizon in which cases arrive and drifts are placed
<code>activity_duration</code>	30, 100	mean and variance of the processing time
<code>waiting_time</code>	15, 50	mean and variance of the waiting time
<code>amount</code>	1,000, 40,000	mean and variance of the amount
<code>seed</code>	42	seed of the random numbers

2. *Draw distributions:* For each activity, Rheon draws a mean duration and a mean waiting time. Each mean lies randomly between 60% and 140% of the mean from Table 4.3, so the activities differ in their times. The variances are the same for all activities. Additionally, Rheon draws a dominant resource for each activity and a dominant region for all cases.
3. *Generate arrivals:* The first case arrives at the start of the time horizon $[t_{min}, t_{max}]$. At each point in time, there is a mean inter-arrival time. The gap to the next case is drawn from an exponential distribution with this mean. Then, the gap to the following case is drawn, and so on, until the next arrival would lie after t_{max} . The mean inter-arrival time starts at $(t_{max} - t_{min})/N$, where N is `num_traces`, so that about N cases arrive. From its drift point on, an arrival rate drift replaces this mean by `inter_arrival` or multiplies it by `factor`, and a workload drift divides it by `workload_factor`. For a gradual drift, the mean moves linearly from its old to its new value across the drift window.
4. *Generate cases:* Each case samples a trace from the pool of the process tree that is valid at its arrival. In the drift window of a gradual control-flow drift, a case samples from the new pool with a probability that grows linearly from 0 to 1. The first event of a case starts at the arrival of the case. For each event, Rheon samples a duration from the normal distribution of its activity and, except for the first event, a waiting time before the event. Each event is executed by the dominant resource of its activity with a probability of 80% and otherwise by a random

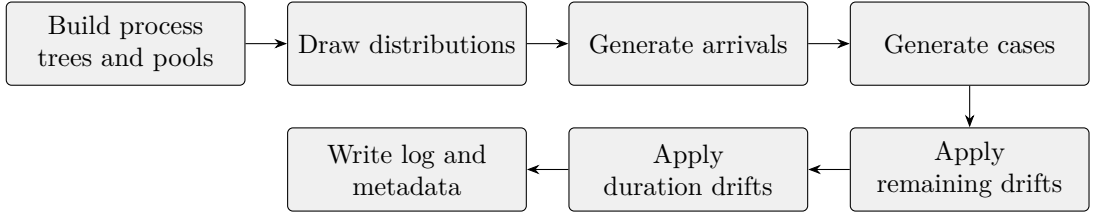


Figure 4.1: The steps in which Rheon generates an event log, in the direction of the arrows. Control-flow drifts take effect when building the process trees and when generating the cases, arrival rate and workload drifts when generating the arrivals. The remaining drifts are reassignment, region, amount, waiting time and pool size drifts.

resource. Similarly, each case receives the dominant region with a probability of 80% and an amount from the normal distribution of the amounts.

5. *Apply the remaining drifts:* The remaining drifts change the generated cases. These are the reassignment, region, amount, waiting time and pool size drifts, which Rheon applies in the order of their start. Drifts on cases, such as region and amount drifts, change the cases that arrive after the drift point. Drifts on events, such as reassignment and waiting time drifts, change the events that start after the drift point. Thus, they also change the later events of cases that arrived before it. For a gradual drift, the change grows linearly across the drift window.
6. *Apply the duration drifts:* Last, the duration drifts multiply the durations of the events of the selected resources by **factor**. Since Rheon applies them after all other drifts, they change exactly the events that a resource executes after all reassignments and pool size changes.
7. *Write log and metadata:* Each event starts after its waiting time from the completion of the previous event of its case and completes after its duration. Rheon computes these start and completion timestamps and rounds them to whole seconds. It then writes the event log as an XES or CSV file together with the metadata file, which also contains the process trees before and after each control-flow drift.

4.3 Drifts

Each drift in **drifts** specifies its drift type **type**, its drift mode **mode**, its position and type-specific parameters. For a sudden drift, the position is a drift point **drift_point**, and for a gradual drift it is a drift window from **start_point** to **end_point**. Positions are fractions of the time horizon between 0 and 1, e.g., the drift point 0.5 lies in the middle of the time horizon. Each drift therefore realizes a process change as in Definition 3.2.1. In a gradual drift, the share of the new behavior grows linearly from 0 to 1 across the drift window. Drifts of the same type must not overlap.

Table 4.4: The nine drift types of Rheon with their perspective and their type-specific parameters.

Perspective	Drift type	What changes	Parameters
Intra-case	<code>control_flow</code>	Cases follow a different process tree.	<code>num_activities</code> , <code>tree_weights</code>
Resource	<code>pool_size</code>	Resources are added or removed, and the durations scale.	<code>delta</code> , <code>duration_factor</code>
Resource	<code>reassignment</code>	Each activity gets a new dominant resource.	–
Resource	<code>workload</code>	The number of cases per unit of time scales.	<code>workload_factor</code>
Resource	<code>duration</code>	The durations of selected resources scale.	<code>resources</code> , <code>factor</code>
Inter-case	<code>waiting_time</code>	The mean waiting time between events shifts.	<code>mean</code> , <code>variance</code>
Inter-case	<code>amount</code>	The amounts follow a shifted distribution.	<code>mean</code> , <code>variance</code>
Inter-case	<code>arrival_rate</code>	The mean inter-arrival time changes.	<code>inter_arrival</code> or <code>factor</code>
Inter-case	<code>region</code>	A new region becomes dominant.	–

Table 4.4 summarizes the nine drift types. They change the generated log as follows:

- *Control-flow* (`control_flow`): Cases after the drift draw their trace from a different process tree, so the trace variants and the directly-follows relations change.
- *Pool size* (`pool_size`): Resources are added or removed. The events of removed resources move to the remaining resources, and each added resource becomes the dominant resource of one activity. The durations are multiplied by `duration_factor` if the pool shrinks and by its reciprocal if the pool grows.
- *Reassignment* (`reassignment`): Each activity gets a new dominant resource.
- *Workload* (`workload`): `workload_factor` times as many cases arrive per unit of time, so each resource handles more or fewer cases.
- *Duration* (`duration`): The durations of the selected resources are multiplied by `factor`.
- *Waiting time* (`waiting_time`): The waiting times between consecutive events of a case follow a new mean or a new variance. A new mean applies to all activities.
- *Amount* (`amount`): The amounts of the cases follow a shifted distribution.
- *Arrival rate* (`arrival_rate`): The mean inter-arrival time becomes `inter_arrival` minutes or is multiplied by `factor`.

4 Rheon

```
import rheon

drifts = [
    {"type": "control_flow", "mode": "gradual",
     "start_point": 0.15, "end_point": 0.35, "num_activities": 9},
    {"type": "reassignment", "mode": "sudden", "drift_point": 0.45},
    {"type": "waiting_time", "mode": "gradual",
     "start_point": 0.55, "end_point": 0.75,
     "mean": 45.0, "variance": 80.0},
    {"type": "arrival_rate", "mode": "sudden", "drift_point": 0.85,
     "factor": 0.5},
]

rheon.generate_log(drifts, "L2.xes", num_traces=10000,
                  num_activities=7, num_resources=7, seed=7)
```

Figure 4.2: Python call that generates the event log L_2 with about 10,000 cases, seven activities and seven resources. The drift positions are fractions of the time horizon.

- *Region* (**region**): A new region becomes the dominant region.

For example, the call in Figure 4.2 generates an event log L_2 with a gradual control-flow drift, a sudden reassignment drift, a gradual waiting time drift and a sudden arrival rate drift.

4.4 Examples

We illustrate one drift per perspective with three example logs (Table 4.5). In each log, the base process tree has five activities in a sequence, and a sudden drift lies in the middle

Table 4.5: The three example logs with their sudden drift on July 1, 2020, and the generated number of cases and events. The control-flow and the reassignment log have 1,600 desired cases, and the arrival rate log has 2,400. The seeds are 107, 202 and 303.

Perspective	Drift type	Change	Cases	Events
Intra-case	control_flow	new process tree $\rightarrow(a, \times(\rightarrow(b, c), \rightarrow(d, e)))$	1,638	6,554
Resource	reassignment	new dominant resource for each activity	1,695	8,475
Inter-case	arrival_rate	mean inter-arrival time from 219 to 4,380 minutes	1,288	6,440

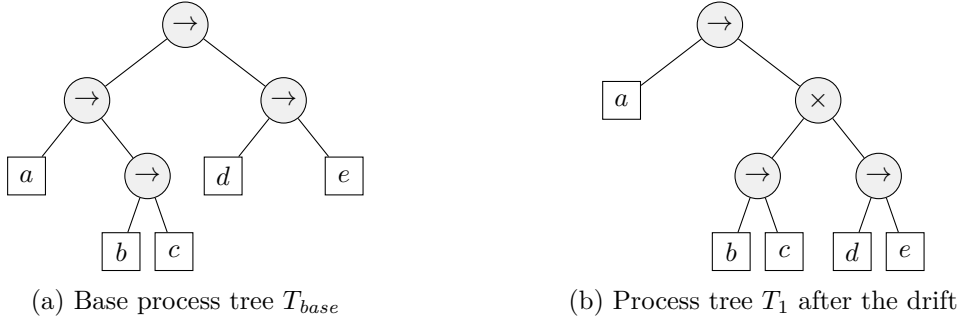


Figure 4.3: The process trees of the example log with a sudden control-flow drift on July 1, 2020. (a) The base process tree $\rightarrow(\rightarrow(a, \rightarrow(b, c)), \rightarrow(d, e))$ produces only the trace $\langle a, b, c, d, e \rangle$. (b) The process tree $\rightarrow(a, \times(\rightarrow(b, c), \rightarrow(d, e)))$ produces the traces $\langle a, b, c \rangle$ and $\langle a, d, e \rangle$.

of the time horizon 2020, i.e., on July 1, 2020. For each example log, a figure shows the log in the process mining tool Celonis. For the control-flow and the reassignment log, the figure also shows a simple measure before and after the drift.

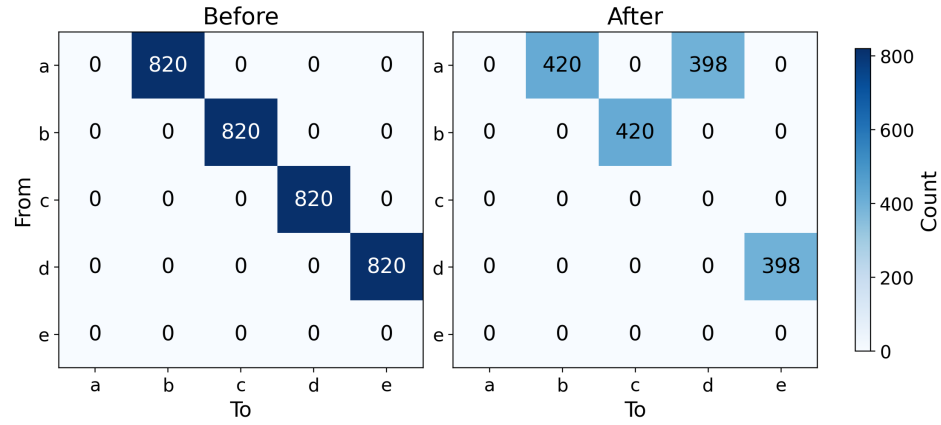
Intra-case perspective: Figure 4.3 shows the process trees of the control-flow drift. Before the drift, all 820 cases follow the trace $\langle a, b, c, d, e \rangle$ of the base process tree T_{base} . From July 1, 2020, the cases follow the process tree T_1 . Therefore, 420 cases follow the trace $\langle a, b, c \rangle$ and 398 cases the trace $\langle a, d, e \rangle$. Figure 4.4a shows the directly-follows counts before and after the drift, and Figures 4.4b and 4.4c show the same log in the variant explorer of Celonis. After the drift, the relation from c to d disappears, and a new relation from a to d appears.

Resource perspective: From July 1, 2020, each activity gets a new dominant resource, e.g., activity a moves from **res_01** to **res_02**. Figure 4.5 shows how many events of each activity each resource executes before and after the drift. Before and after the drift, the dominant resource executes between 84% and 88% of the events of its activity. This matches the expected share of $0.8 + 0.2/3 \approx 0.87$, i.e., 87%, for three resources.

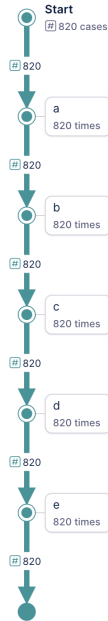
Inter-case perspective: From July 1, 2020, the mean inter-arrival time rises from 219 to 4,380 minutes, so cases arrive 20 times less often. Thus, 1,214 cases arrive before the drift and only 74 cases after it. All cases follow the same trace $\langle a, b, c, d, e \rangle$. Figure 4.6 shows the events of the cases over time in the dotted chart of Celonis. Before the drift, the cases form a steep line, and after the drift, only a few cases arrive.

4.5 Limitations and Future Work

Rheon is a prototype that we develop continuously, so it can still contain bugs. Rheon does not simulate a full operational process. It generates the behavior from process trees and adds times, resources and case attributes to the cases, but it does not model, e.g., the capacities of resources. The generated logs are therefore controlled test data and not replicas of real processes. Compared to CDLG, which also generates incremental and



(a) Directly-follows counts

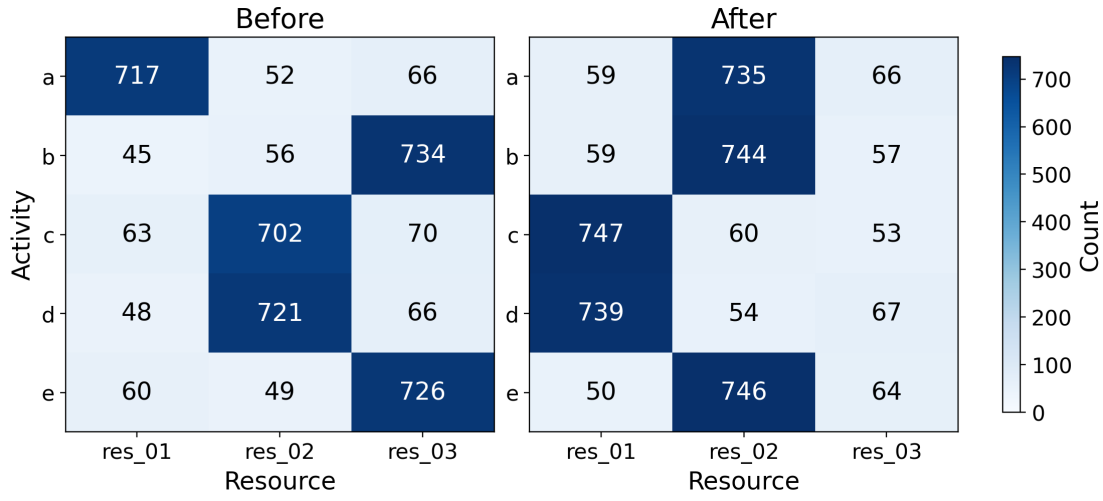


(b) Before the drift

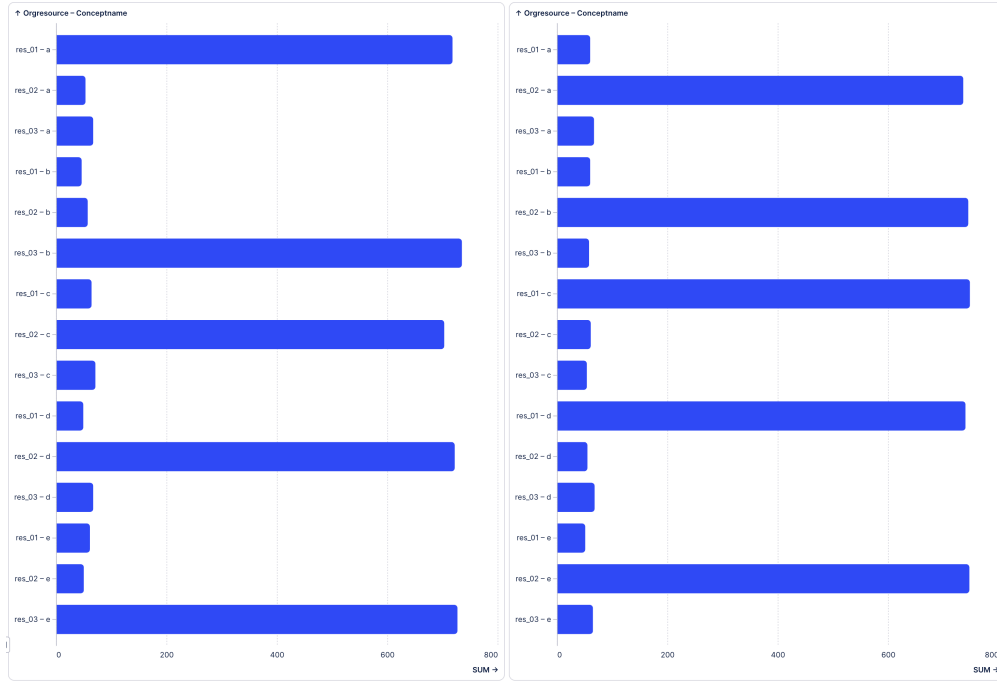


(c) After the drift

Figure 4.4: The example log with a sudden control-flow drift on July 1, 2020. (a) Number of directly-follows relations from an activity (rows) to the next activity (columns) in the cases that arrive before (left) and after (right) the drift. (b) and (c) Screenshots of the variant explorer of Celonis for the cases before and after the drift. The numbers give how often each activity and each directly-follows relation occurs. Before the drift, all cases follow the trace $\langle a, b, c, d, e \rangle$. After the drift, a case follows either $\langle a, b, c \rangle$ or $\langle a, d, e \rangle$.



(a) Events per activity and resource



(b) Events per resource and activity in Celonis

Figure 4.5: The example log with a sudden reassignment drift on July 1, 2020, before (left) and after (right) the drift. (a) Number of events of each activity (rows) that each resource (columns) executes before and after the drift. (b) Screenshot of Celonis with the same numbers as bars for each pair of resource and activity. After the drift, each activity has a new dominant resource, e.g., *a* moves from *res_01* to *res_02*.

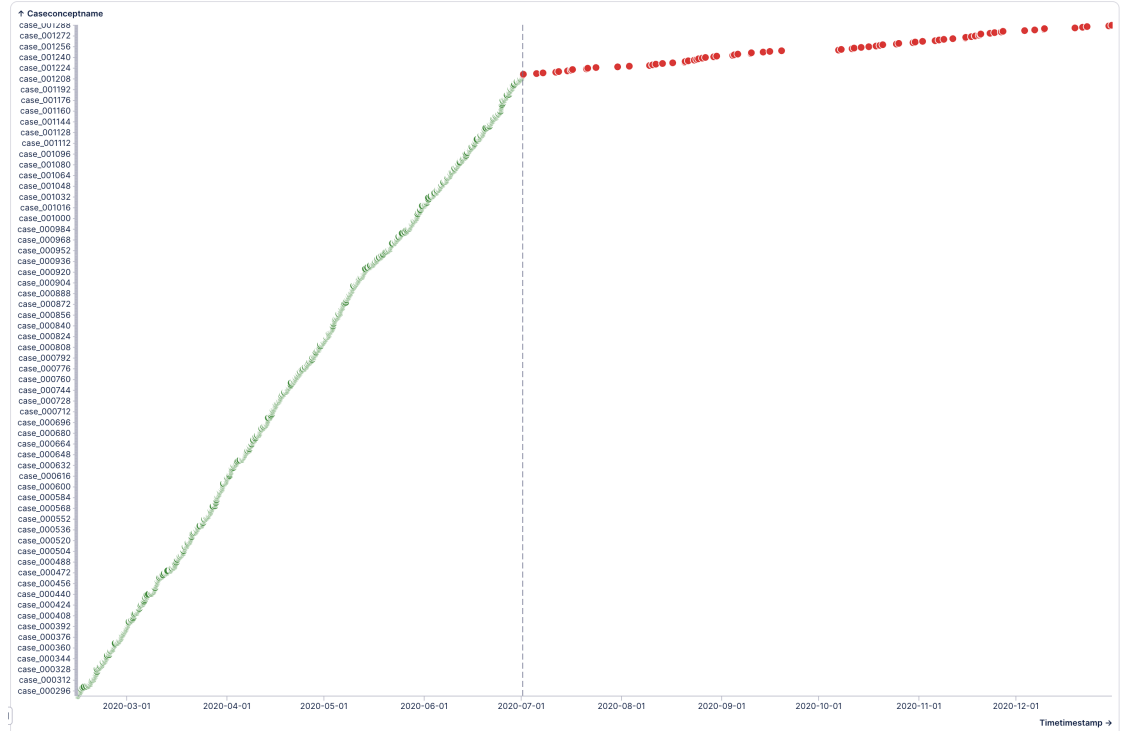


Figure 4.6: Screenshot of the dotted chart of Celonis for the example log with a sudden arrival rate drift on July 1, 2020 (dashed line). Each dot is an event, placed at its time and at its case, with the cases ordered by their arrival. The chart shows the cases from February 2020 on, with the cases before the drift in green and after the drift in red. After the drift, the mean inter-arrival time rises from 219 to 4,380 minutes, so only a few cases arrive.

4.5 Limitations and Future Work

recurring drifts as well as noisy traces [12], Rheon currently supports only sudden and gradual drifts without noise. Future work could extend Rheon with further drift types and drift modes. We also plan to generate object-centric event logs, from which Berti et al. [6] compute execution states.

5 State-Based Drift Detection

Placeholder.

5.1 Feature Extraction

Placeholder.

5.1.1 Intra-Case Perspective

Placeholder.

5.1.2 Resource Perspective

Placeholder.

5.1.3 Inter-Case Perspective

Placeholder.

5.2 Dimensionality Reduction

Placeholder.

5.2.1 Intra-Case Perspective

Placeholder.

5.2.2 Resource Perspective

Placeholder.

5.2.3 Inter-Case Perspective

Placeholder.

5.3 Clustering into States

Placeholder.

5 State-Based Drift Detection

5.3.1 Intra-Case Perspective

Placeholder.

5.3.2 Resource Perspective

Placeholder.

5.3.3 Inter-Case Perspective

Placeholder.

5.4 Drift Signals

Placeholder.

5.4.1 Intra-Case Perspective

Placeholder.

5.4.2 Resource Perspective

Placeholder.

5.4.3 Inter-Case Perspective

Placeholder.

6 Implementation

Placeholder.

6.1 Kairo

Placeholder.

6.2 Dashboard

Placeholder.

7 Evaluation

In this chapter, you present your evaluation. You should have extensively discussed with your supervisor what you are evaluating and why ...

Some typical sections that you need here:

7.1 Experimental Setup

Describe the setup of your experiments. If you apply a pipeline of different techniques, show the pipeline. Present the parameters of your experiments in a table, and, describe them. Typical Elements:

- Input Data Set(s) Used
- Algorithm(s) Used
- Parameter(s) Used
- ...

7.2 Results

Show the results. Try to present your results as structured as you can. A good result (sub)section, follows the following structure:

- Describe what results you are going to Show
- Present an initial hypothesis about the results, e.g., We expect the quality metric M to behave like this, conditional to this parameter P.
- Show the Results
- Confirm the hypothesis.
- If there are results that are not according to the hypothesis, you have to be able to explain why this is the case!!!

7.3 Threats to Validity

Here you discuss any element of your experiments (usually depending on the setup), e.g., data sets used, parameters assessed, that might have implications for the validity of your results. For example, if you have not considered a specific range of parameter values, it is unclear how the algorithm will behave for these settings. If you have excluded certain data, this might have its implications for the generalizability of your results. In a way, this is the discussion section of your thesis.

8 Discussion

In this section, you discuss your general approach. What design decisions did you make, and, what are the implications of this? Maybe your approach computes an exact intermediary result, which is costly in terms of time. Does a suitable approximation approach exist? What are the implications for that approach. You can also dive deeper in very related work and explain why it does not work in certain cases.

9 Conclusion

Your conclusion actually follows the same structure as your abstract. However, you present what the reader has seen, again, using the same concepts as you have already used in your abstract.

Future Work. Furthermore, you present interesting directions for future work!

A Some additional stuff

Try to avoid appendices. If you can't, this is a place to show all kinds of results that are supportive, yet, not critical for your thesis.

Bibliography

- [1] B. R. Abarca Zúñiga, A. Yeshchenko, and H. van der Aa. “Version Clustering: A Top-Down Approach for Process Concept Drift Detection.” In: *Advanced Information Systems Engineering - 38th International Conference, CAiSE 2026, Verona, Italy, June 8-12, 2026, Proceedings, Part I*. Ed. by L. Fuentes, P. Plebani, C. Combi, and H. Reijers. Lecture Notes in Computer Science. Springer, 2026, pp. 61–77. DOI: 10.1007/978-3-032-28110-4_4.
- [2] J. N. Adams, C. Pitsch, T. Brockhoff, and W. M. P. van der Aalst. “An Experimental Evaluation of Process Concept Drift Detection.” In: *Proceedings of the VLDB Endowment* 16.8 (2023), pp. 1856–1869. DOI: 10.14778/3594512.3594517.
- [3] J. N. Adams, S. J. van Zelst, T. Rose, and W. M. P. van der Aalst. “Explainable Concept Drift in Process Mining.” In: *Information Systems* 114 (2023), p. 102177. DOI: 10.1016/j.is.2023.102177.
- [4] E. Alpaydin. *Introduction to Machine Learning*. 2nd ed. Adaptive Computation and Machine Learning. MIT Press, 2010. ISBN: 978-0-262-01243-0.
- [5] S. Barbon Junior, G. M. Tavares, V. G. Turrise da Costa, P. Ceravolo, and E. Damiani. “A Framework for Human-in-the-Loop Monitoring of Concept-Drift Detection in Event Log Stream.” In: *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018*. ACM, 2018, pp. 319–326. DOI: 10.1145/3184558.3186343.
- [6] A. Berti, D. Kretzschmann, and W. M. P. van der Aalst. *Identifying Execution States and Boundary Conditions in Object-Centric Event Logs: A Framework and Case Study*. TechRxiv preprint. 2025. DOI: 10.36227/techrxiv.176522567.74931712/v1.
- [7] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst. “Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science.” In: *Proceedings of the ICPM Demo Track 2019, Co-Located with 1st International Conference on Process Mining (ICPM 2019), Aachen, Germany, June 24-26, 2019*. Ed. by A. Burattin, A. Polyvyanyy, and S. J. van Zelst. Vol. 2374. CEUR Workshop Proceedings. CEUR-WS.org, 2019, pp. 13–16. URL: <https://ceur-ws.org/Vol-2374/paper4.pdf>.
- [8] R. P. J. C. Bose, W. M. P. van der Aalst, I. Žliobaitė, and M. Pechenizkiy. “Handling Concept Drift in Process Mining.” In: *Advanced Information Systems Engineering - 23rd International Conference, CAiSE 2011, London, UK, June 20-24, 2011. Proceedings*. Ed. by H. Mouratidis and C. Rolland. Vol. 6741. Lecture Notes

- in Computer Science. Springer, 2011, pp. 391–405. DOI: 10.1007/978-3-642-21640-4_30.
- [9] T. Brockhoff, M. S. Uysal, and W. M. P. van der Aalst. “Time-Aware Concept Drift Detection Using the Earth Mover’s Distance.” In: *2nd International Conference on Process Mining, ICPM 2020, Padua, Italy, October 5-8, 2020*. IEEE, 2020, pp. 33–40. DOI: 10.1109/ICPM49681.2020.00016.
 - [10] T. Caliński and J. Harabasz. “A Dendrite Method for Cluster Analysis.” In: *Communications in Statistics* 3.1 (1974), pp. 1–27. DOI: 10.1080/03610927408827101.
 - [11] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.” In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), Portland, Oregon, USA*. Ed. by E. Simoudis, J. Han, and U. M. Fayyad. AAAI Press, 1996, pp. 226–231. URL: <https://www.aaai.org/Library/KDD/1996/kdd96-037.php>.
 - [12] J. Grimm, A. Kraus, and H. van der Aa. “CDLG: A Tool for the Generation of Event Logs with Concept Drifts.” In: *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Track at BPM 2022, Münster, Germany, September 11-16, 2022*. Ed. by C. Janiesch, C. Di Francescomarino, T. Grisold, A. Kumar, J. Mendling, B. Pentland, H. Reijers, M. Weske, and R. Winter. Vol. 3216. CEUR Workshop Proceedings. CEUR-WS.org, 2022, pp. 92–96. URL: https://ceur-ws.org/Vol-3216/paper_241.pdf.
 - [13] S. Haykin. *Neural Networks and Learning Machines*. 3rd ed. Pearson Education, 2009. ISBN: 978-0-13-147139-9.
 - [14] S. Khraiweh, D. Dagtekin, and L. Pufahl. “CV-Driven Window Selection for Multi-Perspective Concept Drift Detection in Event Logs.” In: *Business Process Management Forum - BPM 2026 Forum, Toronto, ON, Canada, September 27 - October 2, 2026, Proceedings*. Ed. by F. M. Maggi, H. Leopold, F. Santoro, and A. Gal. Vol. 599. Lecture Notes in Business Information Processing. Springer, 2027, pp. 185–202. DOI: 10.1007/978-3-032-37877-4_11.
 - [15] E. L. Klijn, F. Mannhardt, and D. Fahland. “Multi-Perspective Concept Drift Detection: Including the Actor Perspective.” In: *Advanced Information Systems Engineering - 36th International Conference, CAiSE 2024, Limassol, Cyprus, June 3-7, 2024, Proceedings*. Ed. by G. Guizzardi, F. Santoro, H. Mouratidis, and P. Soffer. Vol. 14663. Lecture Notes in Computer Science. Springer, 2024, pp. 141–157. DOI: 10.1007/978-3-031-61057-8_9.
 - [16] A. Kraus, K. Amiri Elyasi, A. Rebmman, S. Hadian, and H. van der Aa. “A Framework for Steady-State Detection in Process Mining.” In: *Information Systems* 140 (2026), p. 102740. DOI: 10.1016/j.is.2026.102740.
 - [17] A. Kraus and H. van der Aa. “Machine Learning-Based Detection of Concept Drift in Business Processes.” In: *Process Science* 2.1 (2025), p. 5. DOI: 10.1007/s44311-025-00012-w.

- [18] A. Kraus and H. van der Aa. “Comprehensive Characterization of Concept Drifts in Process Mining.” In: *Information Systems* 135 (2026), p. 102584. DOI: 10.1016/j.is.2025.102584.
- [19] A. Maaradji, M. Dumas, M. La Rosa, and A. Ostovar. “Detecting Sudden and Gradual Drifts in Business Processes from Execution Traces.” In: *IEEE Transactions on Knowledge and Data Engineering* 29.10 (2017), pp. 2140–2154. DOI: 10.1109/TKDE.2017.2720601.
- [20] D. Moulavi, P. A. Jaskowiak, R. J. G. B. Campello, A. Zimek, and J. Sander. “Density-Based Clustering Validation.” In: *Proceedings of the 2014 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, 2014, pp. 839–847. DOI: 10.1137/1.9781611973440.96.
- [21] A. Padella and F. Vinci. *Everything Real but the Drift: A Concept-Drift Benchmark Event Logs for Process Mining*. Zenodo dataset, version 1. 2026. DOI: 10.5281/zenodo.21005862.
- [22] P. J. Rousseeuw. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis.” In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.
- [23] D. M. V. Sato, S. C. De Freitas, J. P. Barddal, and E. E. Scalabrin. “A Survey on Concept Drift in Process Mining.” In: *ACM Computing Surveys* 54.9 (2022), 189:1–189:38. DOI: 10.1145/3472752.
- [24] A. Senderovich, C. Di Francescomarino, C. Ghidini, K. Jorbina, and F. M. Maggi. “Intra and Inter-Case Features in Predictive Process Monitoring: A Tale of Two Dimensions.” In: *Business Process Management - 15th International Conference, BPM 2017, Barcelona, Spain, September 10-15, 2017, Proceedings*. Ed. by J. Carmona, G. Engels, and A. Kumar. Vol. 10445. Lecture Notes in Computer Science. Springer, 2017, pp. 306–323. DOI: 10.1007/978-3-319-65000-5_18.
- [25] W. M. P. van der Aalst. *Process Mining: Data Science in Action*. 2nd ed. Springer, 2016. DOI: 10.1007/978-3-662-49851-4.
- [26] W. M. P. van der Aalst. *How to Write Beautiful Process-and-Data-Science Papers?* 2024. arXiv: 2203.09286 [cs.DL]. URL: <https://arxiv.org/abs/2203.09286>.
- [27] C. Zheng, L. Wen, and J. Wang. “Detecting Process Concept Drifts from Event Logs.” In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences - Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Ed. by H. Panetto, C. Debruyne, W. Gaaloul, M. Papazoglou, A. Paschke, C. A. Ardagna, and R. Meersman. Vol. 10573. Lecture Notes in Computer Science. Springer, 2017, pp. 524–542. DOI: 10.1007/978-3-319-69462-7_33.